HỌC MÁY · BÀI 2 · PERCEPTRON

Từ hồi quy tuyến tính đến bài toán phân loại

Mục 1 · Từ “bao nhiêu?” đến “thuộc nhóm nào?”

MỤC TIÊU CỦA MỤC 1

Ba câu hỏi cần trả lời

1

Bài toán phân loại nhị phân được mô tả toán học thế nào?

Dữ liệu, nhãn và quy ước +1 / −1.

2

Vì sao không thể dùng nguyên hồi quy tuyến tính để phân loại?

Một ví dụ nhỏ và nguyên nhân gốc.

3

Ý tưởng hình học nào cho phép phân loại bằng mô hình tuyến tính?

Đường thẳng, siêu phẳng và dấu của điểm số.

NHẮC LẠI TỪ BÀI TRƯỚC

Khung tư duy huấn luyện một mô hình

Mô hình
dự đoán từ đặc trưng

+

Hàm mất mát
đo mức sai của dự đoán

+

Thuật toán tối ưu
cập nhật tham số

=

Huấn luyện
tìm tham số tốt nhất

Hôm nay: giữ biểu thức tuyến tính s = xw, nhưng đổi đầu ra và tiêu chí huấn luyện cho hợp với phân loại.

QUY ƯỚC KÝ HIỆU

Vector đặc trưng viết theo hàng

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu, số đặc trưng	n, d	vô hướng	n quan sát, mỗi mẫu d đặc trưng
Vector đặc trưng (hàng)	$\mathbf{x}_i$	1 × (d+1)	mẫu thứ i, phần tử đầu bằng 1
Ma trận đặc trưng	**X**	n × (d+1)	hàng thứ i của X là x_i
Vector tham số (cột)	**w**	(d+1) × 1	hệ số chệch w_0, trọng số $w_1 \ldots w_d$
Vector nhãn thực (cột)	**y**	n × 1	$y_i \in \{-1, +1\}$
Điểm số	$s_i = x_i w$	s = Xw: n × 1	giá trị tuyến tính của mẫu i
Nhãn dự đoán	$\hat{\mathbf{y}}_i$	n × 1	$\hat{y}_i \in \{-1, +1\}$

So với bài trước: x viết theo hàng nên s = xw, còn trước đây $s = w^T x$. Hai cách cho cùng một giá trị.

1.1

Bài toán phân loại nhị phân

Dữ liệu, nhãn và quy ước +1 / −1.

Khác nhau ở loại đầu ra cần dự đoán

HỒI QUY

Bao nhiêu?

- Giá một căn nhà
- Nhiệt độ ngày mai
- Điểm thi

Đầu ra: một số thực

PHÂN LOẠI

Thuộc nhóm nào?

- Thư rác hay thư thường
- Đậu hay rớt
- Bệnh hay không bệnh

Đầu ra: một nhãn trong tập hữu hạn

1.1 · PHÂN LOẠI NHỊ PHÂN

Hai nhóm: lớp dương và lớp âm



Lớp dương: y = +1

positive class · ví dụ: sinh viên đậu



Lớp âm: y = −1

negative class · ví dụ: sinh viên rớt

“Dương” và “âm” chỉ là tên gọi, không có nghĩa là tốt hay xấu.

Thường lớp dương là lớp ta quan tâm cần phát hiện, ví dụ thư rác hay bệnh nhân mắc bệnh.

1.1 · MÔ TẢ BÀI TOÁN

Bộ dữ liệu huấn luyện gồm n quan sát

$$D = \{ (\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_n, y_n) \}, \; y_i \in \{-1, +1\}$$

$\mathbf{x}_i$

Vector đặc trưng (hàng) của quan sát thứ i.

y_i

Nhãn thực tế, chỉ nhận một trong hai giá trị −1 hoặc +1.

Mục tiêu: tìm quy tắc dự đoán $\hat{y} \in \{-1, +1\}$ từ x, sao cho ŷ trùng với y trên càng nhiều mẫu càng tốt.

1.1 · VÍ DỤ XUYÊN SUỐT

Dự đoán sinh viên đậu hay rớt

SV	x_1 giờ học	x_2 buổi vắng	y
1	2	4	−1 rớt
2	3	3	−1 rớt
3	4	5	−1 rớt
4	6	1	+1 đậu
5	8	2	+1 đậu
6	9	0	+1 đậu

n = 6, d = 2

Thêm cột 1 vào đặc trưng: $\mathbf{x_i} = [1, x_{i1}, x_{i2}]$.

Ví dụ sinh viên 4: $\mathbf{x_4} = [1, 6, 1]$.

Mỗi hàng của ma trận X là vector đặc trưng của một sinh viên.

1.1 · QUY ƯỚC NHÃN

Dùng nhãn {−1, +1} thay vì {0, 1}

Chỉ là hai cách mã hoá của cùng một ý nghĩa. Đổi qua lại: $y' = 2y - 1$

y	ŷ	y·ŷ	Ý nghĩa
+1	+1	+1	Dự đoán đúng
−1	−1	+1	Dự đoán đúng
+1	−1	−1	Sai: bỏ sót mẫu lớp dương
−1	+1	−1	Sai: báo nhầm thành lớp dương

Vì sao chọn ±1?

Nhãn dự đoán lấy trực tiếp từ dấu của điểm số.

Đúng hay sai gói trong một phép nhân: y·ŷ = +1 là đúng, −1 là sai.

1.2

Vì sao không dùng thẳng hồi quy tuyến tính?

Một ý tưởng tự nhiên, một phản ví dụ nhỏ.

1.2 · MỘT Ý TƯỞNG TỰ NHIÊN

Coi nhãn −1 và +1 như hai con số

1. **Huấn luyện hồi quy tuyến tính (MSE) trên nhãn ±1.**
2. **Thu được điểm số** $s = xw = w_0 + w_1x_1 + \ldots + w_d x_d$.
3. **Đặt ngưỡng ở giữa hai nhãn, tức là 0.**

$$\hat{y} = +1 \text{ nếu } s \geq 0$$
$$\hat{y} = -1 \text{ nếu } s < 0$$

Nghe hợp lý. Hãy thử trên 8 sinh viên.

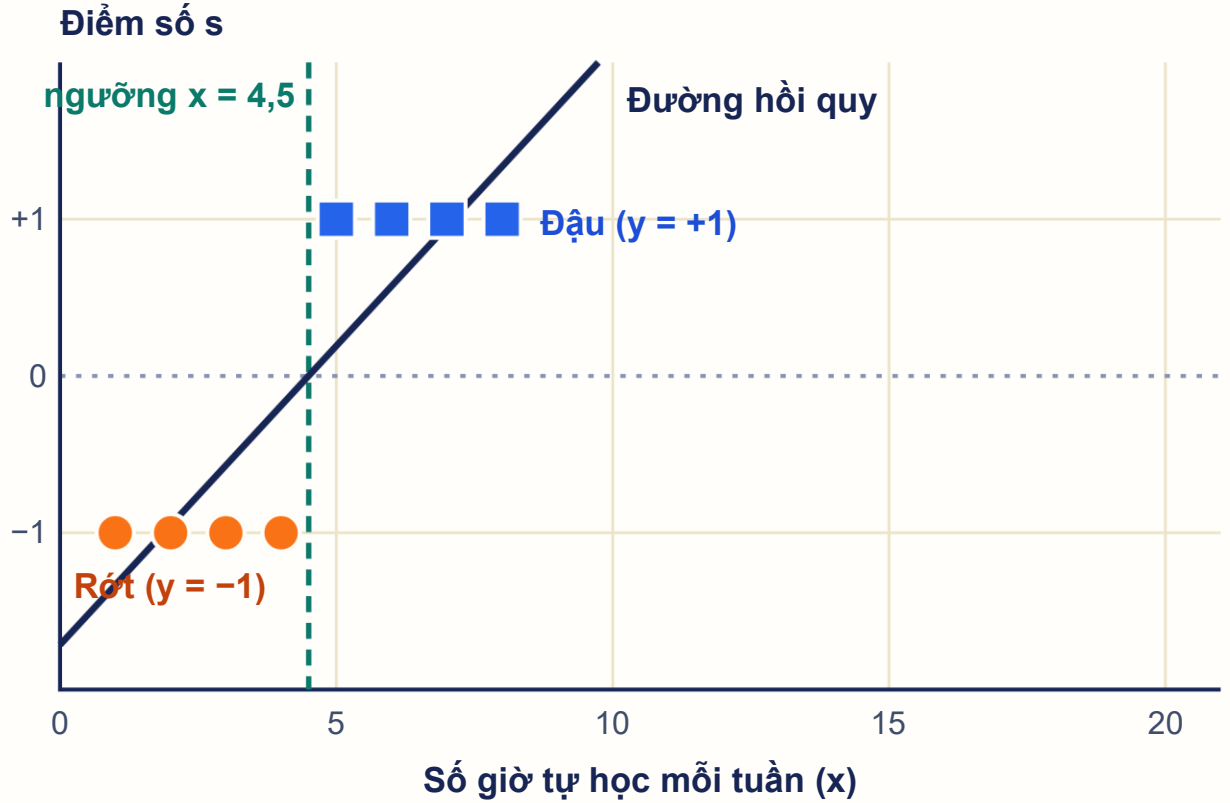
Học 1, 2, 3, 4 giờ: **rớt** (y = −1).

Học 5, 6, 7, 8 giờ: **đậu** (y = +1).

Chỉ một đặc trưng: số giờ tự học x.

1.2 · VÍ DỤ MỘT ĐẶC TRƯNG

Quan sát 1: đầu ra không phải là nhãn



s = −1,714 + 0,381·x, ngưỡng s = 0 tại x = 4,5.

Phân loại đúng cả 8 sinh viên. Đến đây mọi thứ vẫn ổn.

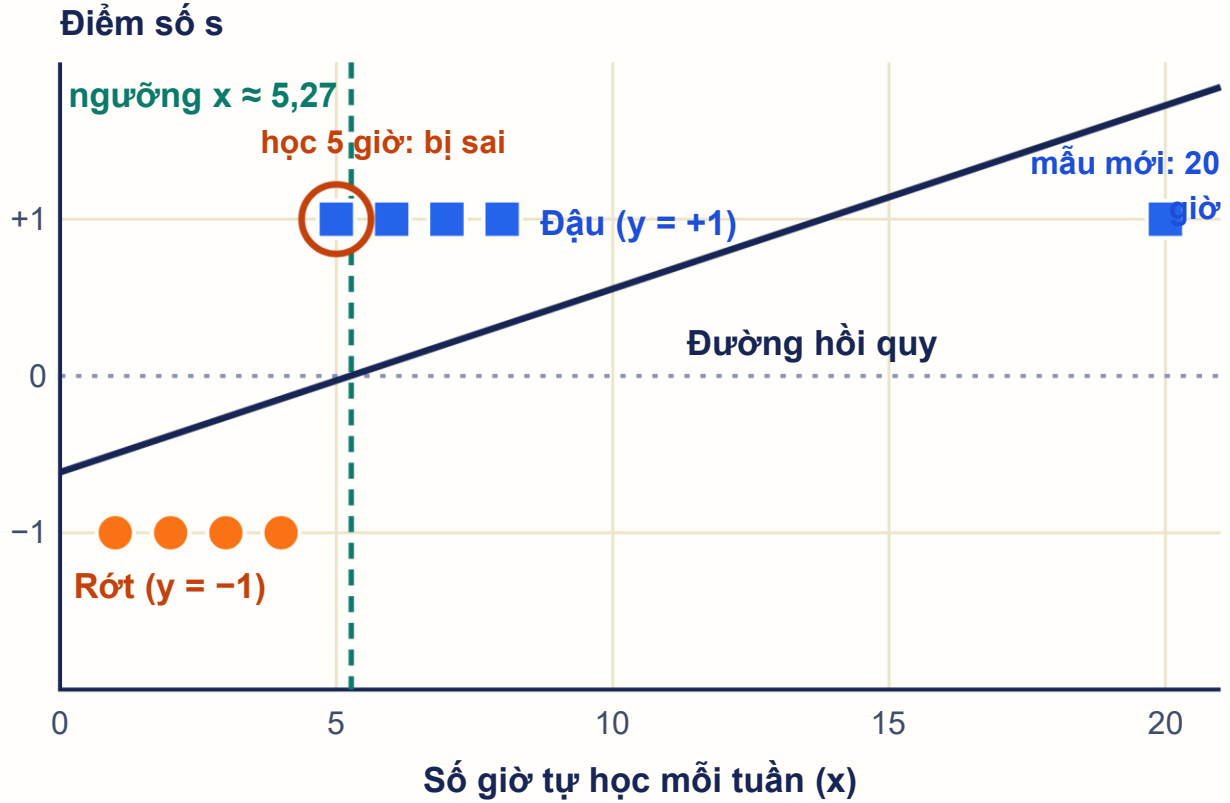
x = 1 → s ≈ −1,33

x = 8 → s ≈ +1,33

Hai giá trị này nằm ngoài [−1, 1], không có ý nghĩa như một nhãn. Nhãn chỉ xuất hiện nhờ bước đặt ngưỡng thủ công.

1.2 · THÊM MỘT SINH VIÊN HỌC 20 GIỜ

Quan sát 2: một mẫu “rất đúng” làm hỏng mô hình



s = −0,614 + 0,117·x, ngưỡng dịch sang x ≈ 5,27.

Mẫu mới hoàn toàn phù hợp xu hướng chung, ta mong nó không gây rắc rối.

Học 5 giờ: s ≈ −0,03 < 0

⇒ dự đoán rớt, nhưng thực tế đậu

Mẫu này trước đó được phân loại đúng. Chỉ vì thêm một mẫu dễ mà nó thành sai.

1.2 · NGUYÊN NHÂN

Hàm mất mát không đo đúng điều ta quan tâm

MSE ĐO

Khoảng cách từ s tới −1 hoặc +1

Cố đưa s càng gần nhãn càng tốt.

≠

TA QUAN TÂM

Số mẫu được phân loại đúng

Chỉ cần s nằm đúng phía của ngưỡng.

Sinh viên học 20 giờ:

s ≈ 1,72: đã ở phía "đậu", phân loại đúng → **MSE vẫn phạt vì 1,72 ≠ 1** → **Mô hình hạ độ dốc, các mẫu khác lệch ngưỡng**

Với nhiều hơn hai lớp, gán nhãn 1, 2, 3 còn vô tình áp đặt thứ tự giữa các lớp.

1.2 · KẾT LUẬN

Hồi quy không sai, chỉ không được thiết kế cho phân loại

	Hồi quy tuyến tính	Phân loại cần
Đầu ra	Một số thực s	Một nhãn $\hat{y} \in \{-1, +1\}$
Câu hỏi	Bao nhiêu?	Thuộc nhóm nào?
Tiêu chí	Khoảng cách $\lvert y - s \rvert$ nhỏ	Đúng nhãn càng nhiều càng tốt

1 **Mô hình cho ra nhãn ngay từ đầu, không đặt ngưỡng thủ công**

2 **Tiêu chí huấn luyện phản ánh đúng hay sai của dự đoán**

Tin vui: biểu thức tuyến tính $s = xw$ vẫn dùng được. Mục 1.3 cho thấy ý nghĩa hình học của nó.

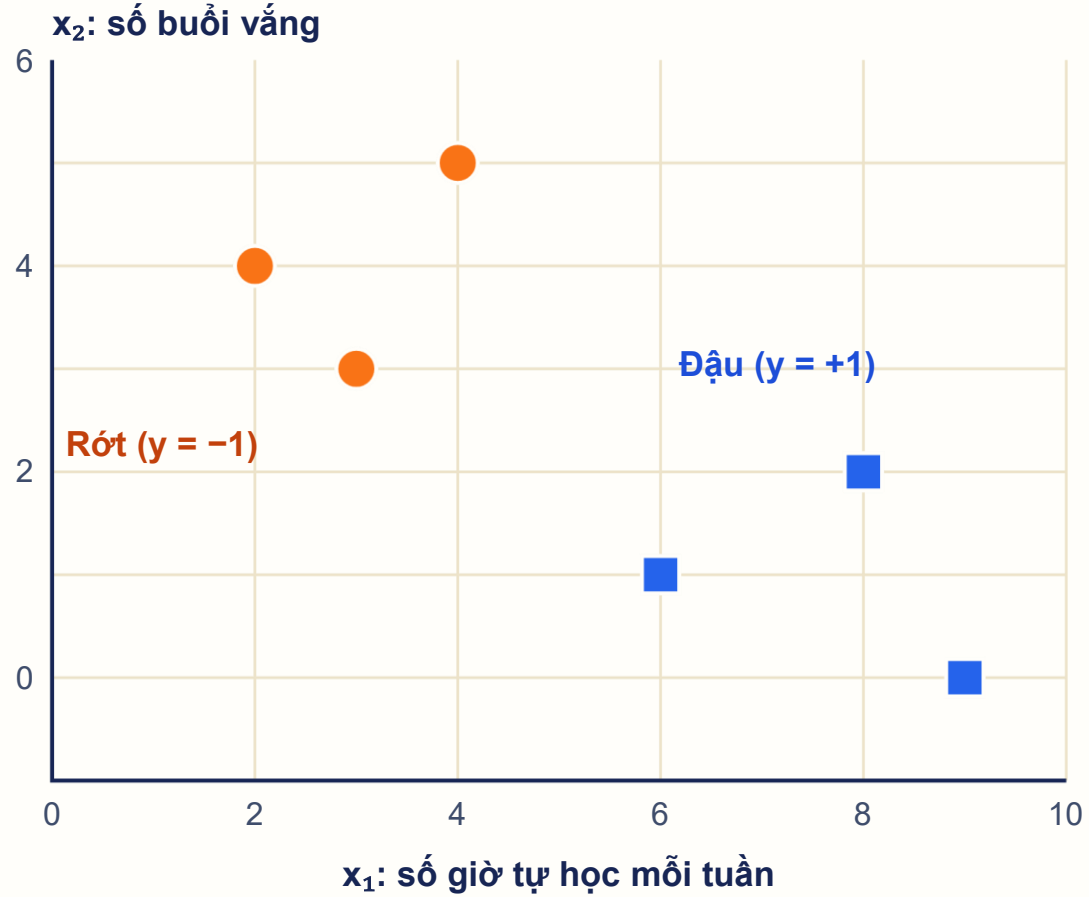
1.3

Phân loại bằng một đường thẳng hoặc siêu phẳng

Ý tưởng hình học của phân loại tuyến tính.

1.3 · NHÌN DỮ LIỆU NHƯ CÁC ĐIỂM

Mỗi sinh viên là một điểm trên mặt phẳng



d đặc trưng ⇒ một điểm trong không gian d chiều

Ở đây d = 2: mỗi sinh viên là một điểm (x_1, x_2).

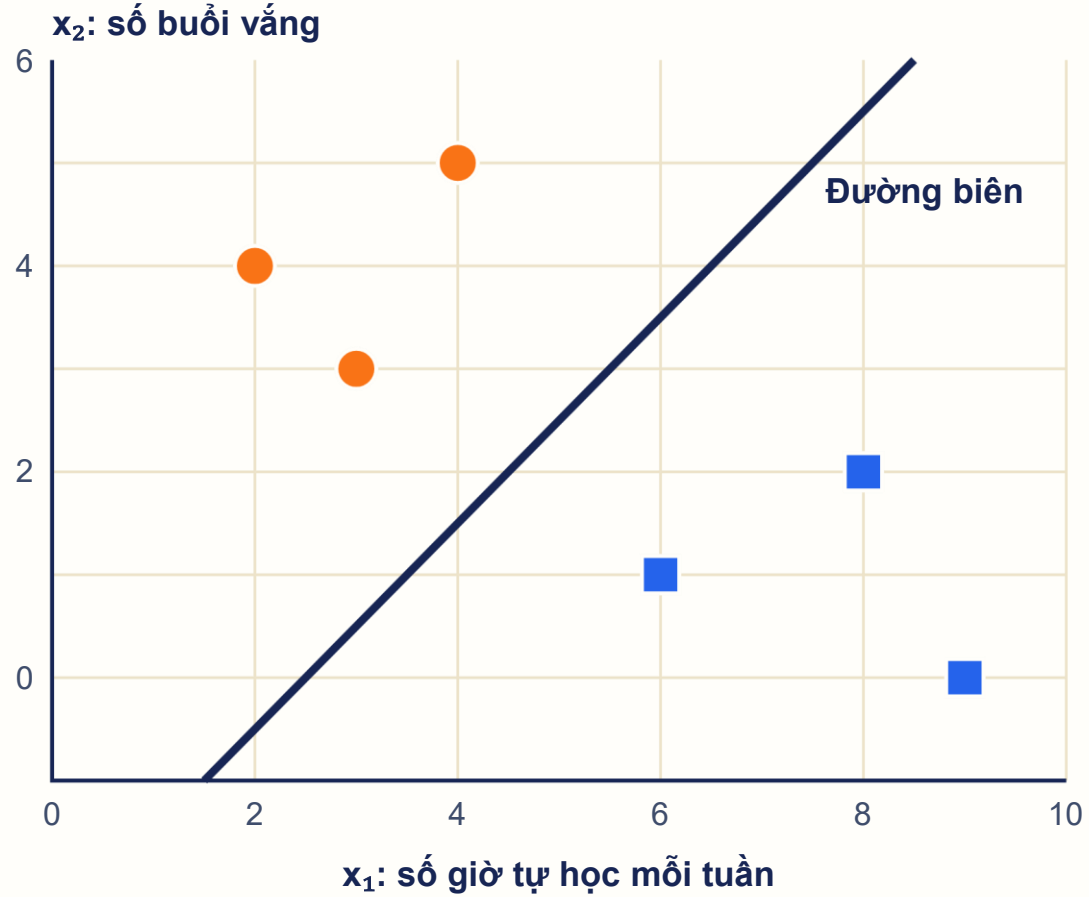
Quan sát bằng mắt

Sinh viên rớt nằm phía trên bên trái, sinh viên đậu nằm phía dưới bên phải.

Hai nhóm có thể được ngăn cách bằng một đường thẳng.

1.3 · ĐƯỜNG BIÊN QUYẾT ĐỊNH

Phương trình của đường biên



$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

$$\Leftrightarrow \mathbf{xw} = \boldsymbol{0}$$

d = 1: một điểm

d = 2: một đường thẳng

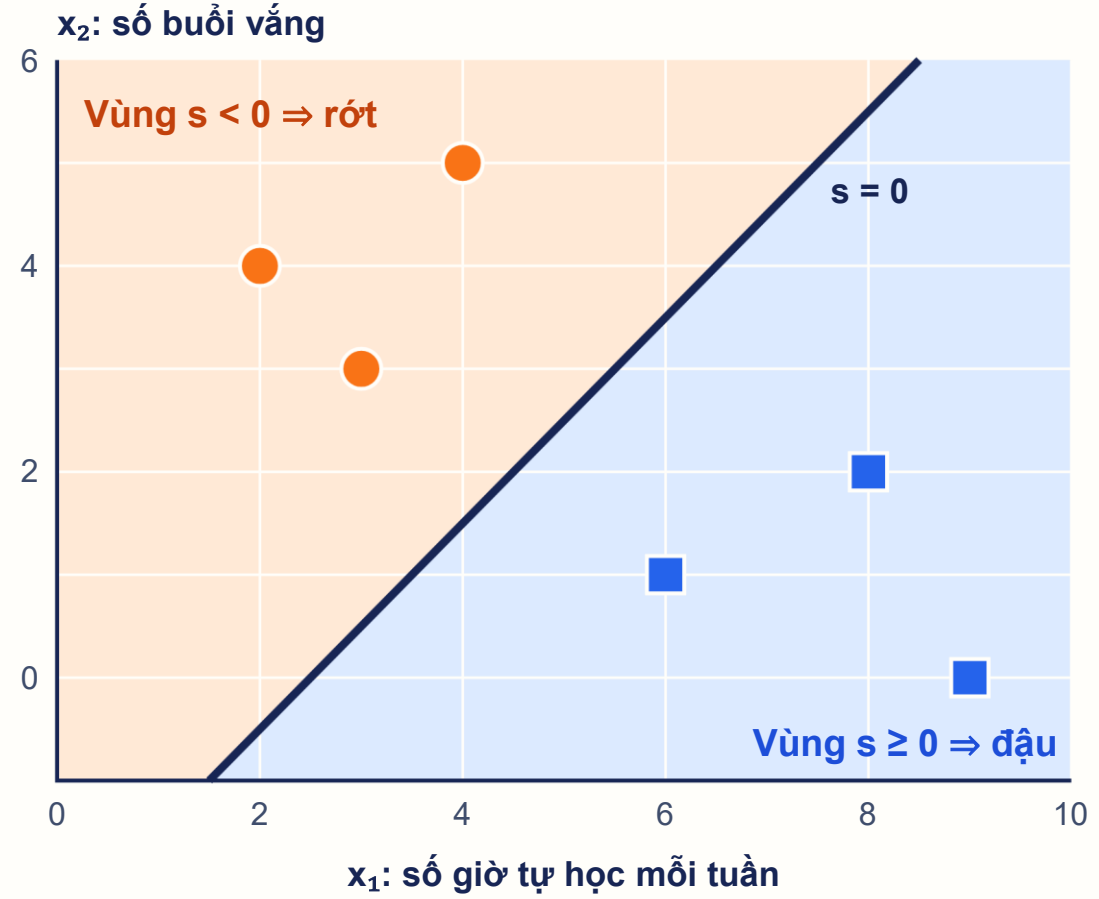
d = 3: một mặt phẳng

d > 3: siêu phẳng (hyperplane), không vẽ được

xw chính là biểu thức của hồi quy tuyến tính. Khác ở cách dùng: nó cho biết điểm nằm ở phía nào của đường biên.

1.3 · ĐIỂM SỐ

Dấu của điểm số cho biết nhãn



$$s = \mathbf{x}\mathbf{w}$$

s > 0	Một phía của đường biên: ŷ = +1
s < 0	Phía còn lại: ŷ = −1
s = 0	Điểm nằm ngay trên đường biên

Ngưỡng 0 đã tích hợp sẵn. Hệ số chệch w_0 dịch đường biên, nên không cần chọn ngưỡng riêng.

1.3 · VÍ DỤ SỐ

Chọn **w** = $[-2,5; 1; -1]^T$ và tính s = **Xw**

SV	s	ŷ	y	y·s	Kết quả
1	−4,5	−1	−1	4,5	Đúng
2	−2,5	−1	−1	2,5	Đúng
3	−3,5	−1	−1	3,5	Đúng
4	2,5	+1	+1	2,5	Đúng
5	3,5	+1	+1	3,5	Đúng
6	6,5	+1	+1	6,5	Đúng

Đường biên: $-2,5 + x_1 - x_2 = 0$, tức $x_2 = x_1 - 2,5$.

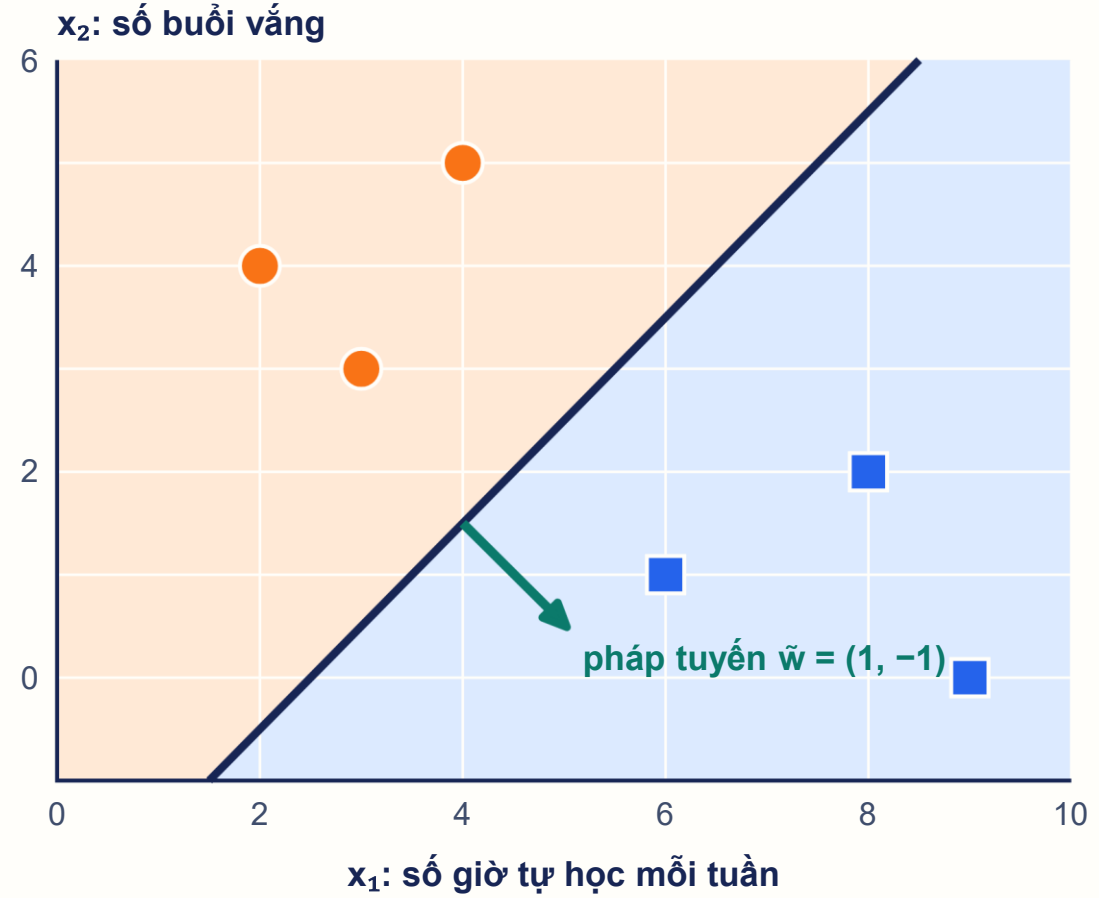
Phân loại đúng cả 6 sinh viên, không cần ngưỡng thủ công.

$y \cdot s > 0 \Leftrightarrow$ đúng

Dấu của y·s cho biết đúng hay sai. Độ lớn cho biết đúng (hoặc sai) nhiều đến mức nào.

1.3 · Ý NGHĨA HÌNH HỌC CỦA THAM SỐ

Pháp tuyến quyết định hướng, w_0 quyết định vị trí



$\tilde{\mathbf{w}} = [w_1 \ \ldots \ w_d]^T$ là pháp tuyến

Vuông góc với đường biên và chỉ về phía điểm số s tăng. Ở đây mũi tên chỉ về phía các sinh viên đậu.

w_0 quyết định vị trí

Đổi w_0: đường biên tịnh tiến song song với chính nó.

Đổi $\tilde{\mathbf{w}}$: đường biên xoay

Hướng pháp tuyến thay đổi nên đường biên quay theo.

1.3 · KHOẢNG CÁCH TỚI ĐƯỜNG BIÊN

Càng xa đường biên, càng tự tin về nhãn

$$\text{khoảng cách} = |s| \,/\, \|\tilde{\mathbf{w}}\| = |\mathbf{xw}| \,/\, \sqrt{(w_1^2 + \ldots + w_d^2)}$$

Ví dụ: sinh viên 4

$s = 2{,}5$ và $\|\tilde{w}\| = \sqrt{(1^2 + (-1)^2)} = \sqrt{2}$

$$\text{Khoảng cách} = 2{,}5 \,/\, \sqrt{2} \approx 1{,}77$$

Khoảng cách có dấu

$$y_i s_i \,/\, \|\tilde{\mathbf{w}}\|$$

Dương: nằm đúng phía. Âm: nằm sai phía.

Điểm sát đường biên chỉ cần dịch chuyển một chút là đổi nhãn. Điểm càng xa (về phía đúng) thì dự đoán càng chắc chắn.

1.3 · PHÂN TÁCH TUYẾN TÍNH ĐƯỢC

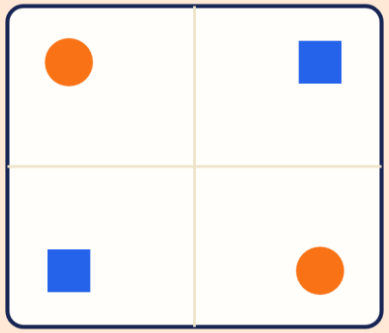
Khi nào một đường thẳng là đủ?

Dữ liệu **phân tách tuyến tính được** nếu tồn tại w sao cho mọi mẫu nằm đúng phía:

$$y_i \, (\mathbf{x}_i \mathbf{w}) > 0, \text{ mọi } i = 1, \ldots, n$$

Dùng bất đẳng thức chặt: mẫu nằm đúng trên đường biên (s = 0) không được coi là đúng phía chắc chắn.

Dữ liệu sinh viên thoả điều kiện này, vì cột y·s đều dương.

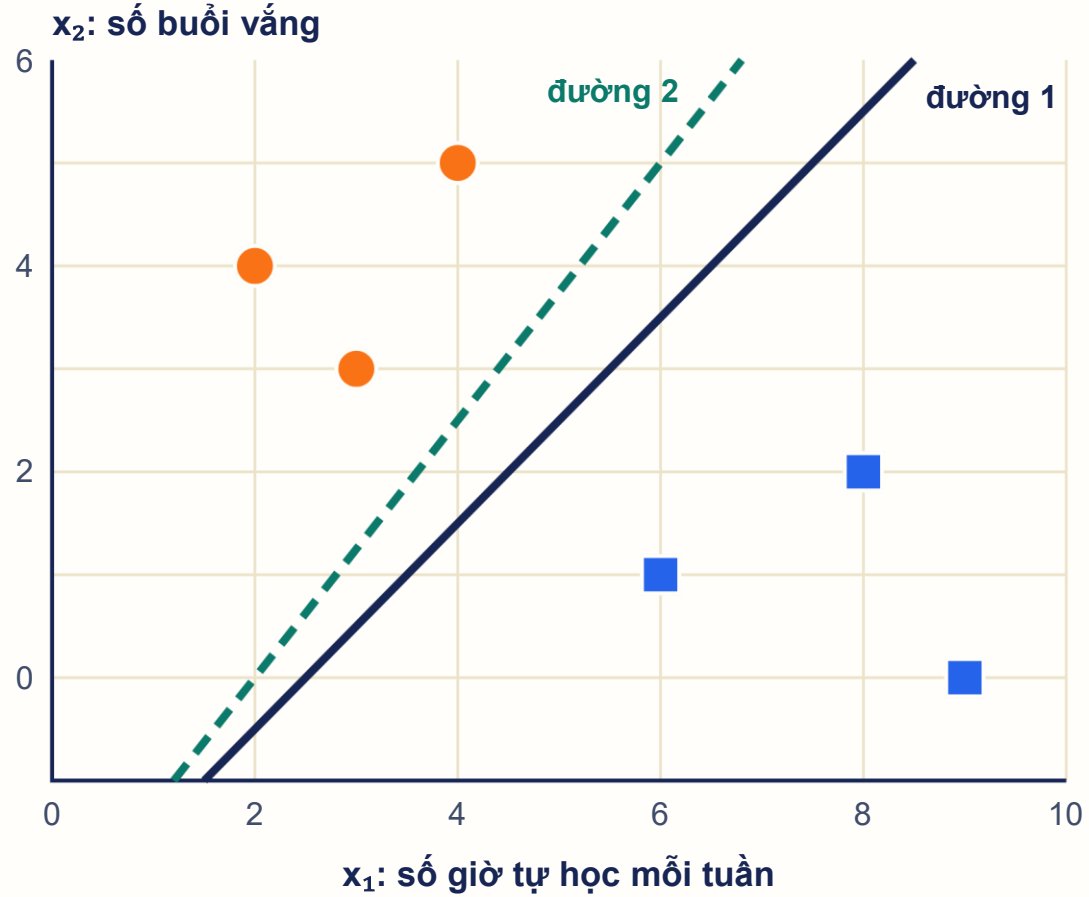


Bài toán XOR

Không có đường thẳng nào tách được hai nhóm: giới hạn cơ bản của mô hình tuyến tính.

1.3 · CHỌN ĐƯỜNG NÀO?

Có vô số đường thẳng cùng phân loại đúng



Đường 1: w = $[-2{,}5;\ 1;\ -1]^T$

Cả 6 mẫu đều có y·s > 0.

Đường 2: w′ = $[-2;\ 1;\ -0{,}8]^T$

y·s = 3,2; 1,4; 2; 3,2; 4,4; 7, đều dương.

w lấy ở đâu?

Cần một quá trình huấn luyện: bắt đầu từ w khởi tạo rồi cải thiện dần, giống Gradient Descent ở bài trước.

Mô hình thực hiện ý tưởng này: Perceptron (mục 2 và mục 3).

TÓM TẮT MỤC 1

Năm ý cần nhớ

1. **Phân loại nhị phân** dự đoán nhãn $y \in \{-1, +1\}$. Tích $y \cdot \hat{y} = +1$ khi đúng và -1 khi sai.

2. **Hồi quy + ngưỡng** không phù hợp: đầu ra không phải nhãn, và MSE đo khoảng cách chứ không đo đúng sai.

3. **Ý tưởng tuyến tính:** siêu phẳng $\mathbf{xw} = 0$ chia không gian thành hai phía. Dấu của $s = \mathbf{xw}$ cho nhãn, dấu của $y \cdot s$ cho biết đúng hay sai.

4. **$\tilde{w}$ là pháp tuyến** của đường biên, còn w_0 **quyết định vị trí**.

5. **Phân tách tuyến tính được** khi $y_i \, \mathbf{x}_i \mathbf{w} > 0$ với mọi i. Có vô số **w** như vậy, nên cần thuật toán huấn luyện để tìm ra một w.

KIỂM TRA NHANH

Năm câu hỏi cho sinh viên

1. Với $\mathbf{w} = [-2{,}5;\ 1;\ -1]^T$ và $\mathbf{x} = [1,\ 5,\ 2]$, tính s và ŷ.

2. Mẫu có y = −1 nhưng ŷ = +1. Tích y·ŷ bằng bao nhiêu? Đây là loại sai nào?

3. Sinh viên 2 ($\mathbf{x} = [1,\ 3,\ 3]$, y = −1), cùng w ở câu 1: tính s và y·s. Đúng hay sai?

4. Giải thích: vì sao thêm một mẫu “hiển nhiên đúng” (học 20 giờ, đậu) lại làm hồi quy phân loại sai một mẫu khác?

5. Đường $2x_1 - 2x_2 - 5 = 0$ có cùng đường biên với $x_1 - x_2 - 2{,}5 = 0$ không? Vector $[-5,\ 2,\ -2]^T$ có cho cùng nhãn dự đoán trên 6 sinh viên không?

GỢI Ý ĐÁP ÁN

Đáp án kiểm tra nhanh

1 $s = -2{,}5 + 5 - 2 = 0{,}5 \geq 0$ nên $\hat{y} = +1$ (dự đoán đậu).

2 $y \cdot \hat{y} = (-1)(+1) = -1$. Lỗi báo nhầm: mẫu lớp âm bị dự đoán thành lớp dương.

3 $s = -2{,}5 + 3 - 3 = -2{,}5$ và $y \cdot s = 2{,}5 > 0$. Mô hình đúng trên mẫu này.

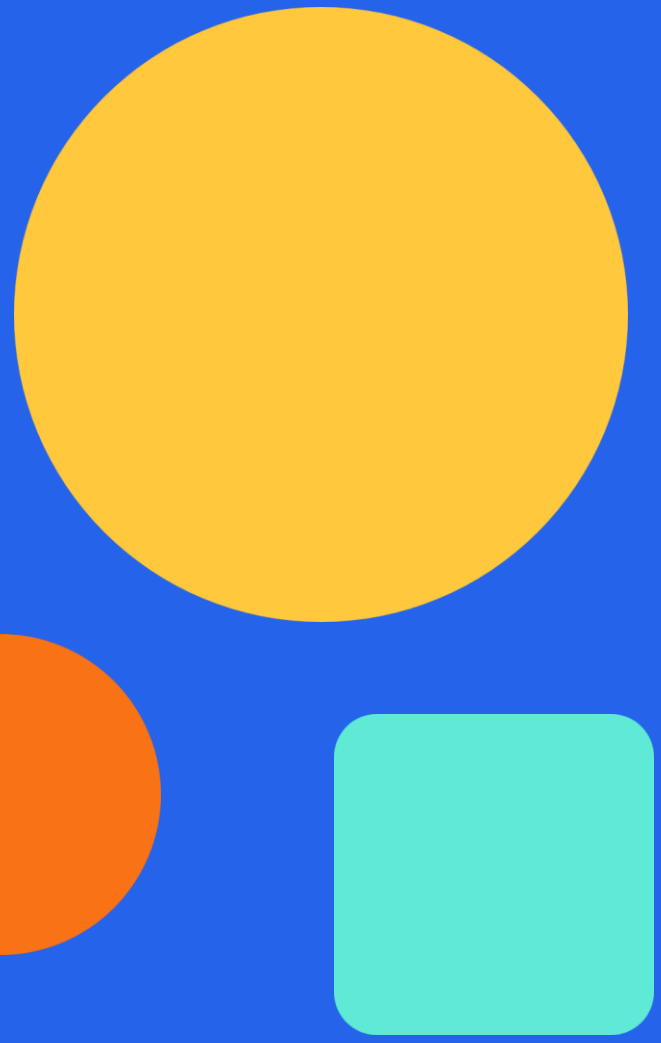
4 Điểm số của mẫu mới đã đúng phía ngưỡng nhưng MSE vẫn phạt vì s khác 1. Đường hồi quy bị kéo thấp xuống, ngưỡng dịch sang phải, và sinh viên học 5 giờ rơi xuống phía “rớt”.

5 Có. Phương trình thứ hai bằng phương trình thứ nhất nhân 2. Điểm số mới gấp đôi điểm số cũ, cùng dấu, nên nhãn không đổi. Vậy w và $c \cdot w$ ($c > 0$) xác định cùng một bộ phân loại.

HỌC MÁY · BÀI 2 · PERCEPTRON

Mô hình Perceptron

Mục 2 · Từ điểm số tuyến tính đến một quyết định



MỞ ĐẦU MỤC 2

Perceptron: nơ-ron nhân tạo đầu tiên biết học

1958

Frank Rosenblatt đề xuất Perceptron

Rất đơn giản

Tính một tổng có trọng số rồi lấy dấu.

Có thể học từ dữ liệu

Tham số được cải thiện dần qua huấn luyện (mục 3).

Là nền tảng

Của Logistic Regression và các mạng nơ-ron hiện đại.

MỤC TIÊU CỦA MỤC 2

Ba câu hỏi cần trả lời

1

Perceptron nhận đầu vào, xử lý và ra quyết định thế nào?

Mục 2.1: tổng có trọng số, hệ số chệch, hàm dấu.

2

Mô hình được viết gọn bằng vector và ma trận ra sao?

Mục 2.2: tính cho cả tập dữ liệu và đại lượng margin.

3

Perceptron chia không gian đặc trưng thế nào, tham số điều khiển ra sao?

Mục 2.3: đường biên, tịnh tiến và xoay.

2.1

Tổng có trọng số, hệ số chệch và hàm dấu

Một “máy ra quyết định” ba bước.

2.1 · Ý TƯỞNG HOẠT ĐỘNG

Cân nhắc nhiều yếu tố rồi quyết định

1

Nhận đặc trưng

Đầu vào là x_1, x_2, …, x_d của một mẫu.

2

Tính điểm số

Nhân mỗi đặc trưng với trọng số (mức quan trọng), cộng lại, rồi cộng một hằng số điều chỉnh.

3

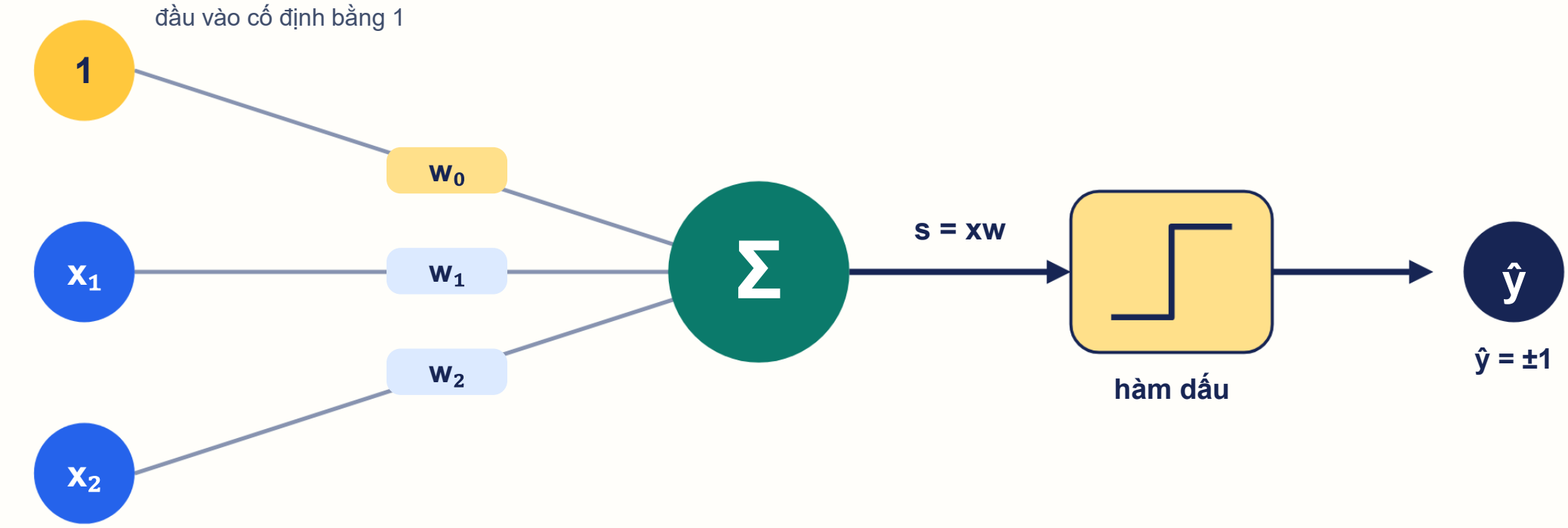
Ra quyết định

Điểm số không âm: lớp dương (ŷ = +1). Ngược lại: lớp âm (ŷ = −1).

Giống một người ra quyết định: mỗi yếu tố có mức quan trọng riêng, cộng lại rồi quyết.

2.1 · CẤU TRÚC PERCEPTRON VỚI HAI ĐẶC TRƯNG

Đầu vào × trọng số → điểm số → hàm dấu → nhãn



1. Nhận đặc trưng

2. Tính điểm số

3. Ra quyết định

2.1 · ĐIỂM SỐ

Tổng có trọng số cộng hệ số chệch

$$s = w_0 + w_1x_1 + w_2x_2 + \ldots + w_dx_d = \mathbf{xw}$$

$$\mathbf{x} = [1, x_1, \ldots, x_d], \mathbf{w} = [w_0, w_1, \ldots, w_d]^T$$

$w_j > 0$

x_j tăng thì s tăng.

$w_j < 0$

x_j tăng thì s giảm.

$w_j = 0$

x_j không có ảnh hưởng.

Hệ số chệch w_0 **là điểm số khi mọi đặc trưng bằng 0: "mức khởi điểm" của mô hình.**

2.1 · ĐỌC CÔNG THỨC BẰNG LỜI

Ví dụ sinh viên với $\mathbf{w} = [-2{,}5;\ 1;\ -1]^T$

$$s = -2{,}5 + 1 \cdot x_1 - 1 \cdot x_2$$

+1 cho mỗi giờ tự học

Học thêm 1 giờ thì s tăng 1.

−1 cho mỗi buổi vắng

Vắng thêm 1 buổi thì s giảm 1.

Khởi điểm −2,5

Phải tích lũy đủ giờ học thì mới có s ≥ 0.

w_0 quyết định độ khó của việc được xếp vào lớp dương.

Biểu thức s = **xw** giống hệt hồi quy tuyến tính. Khác biệt nằm ở bước sau: đưa s qua một hàm kích hoạt.

2.1 · HÀM KÍCH HOẠT DẤU

Từ điểm số thực đến nhãn ±1

Hàm dấu (sign)

$$\text{sign}(s) = +1 \text{ nếu } s \geq 0$$

$$\text{sign}(s) = -1 \text{ nếu } s < 0$$

Quy tắc dự đoán

$$\hat{y} = \text{sign}(s) = \text{sign}(\mathbf{xw})$$

xw ≥ 0: lớp dương. **xw** < 0: lớp âm.

NHỚ GỌN

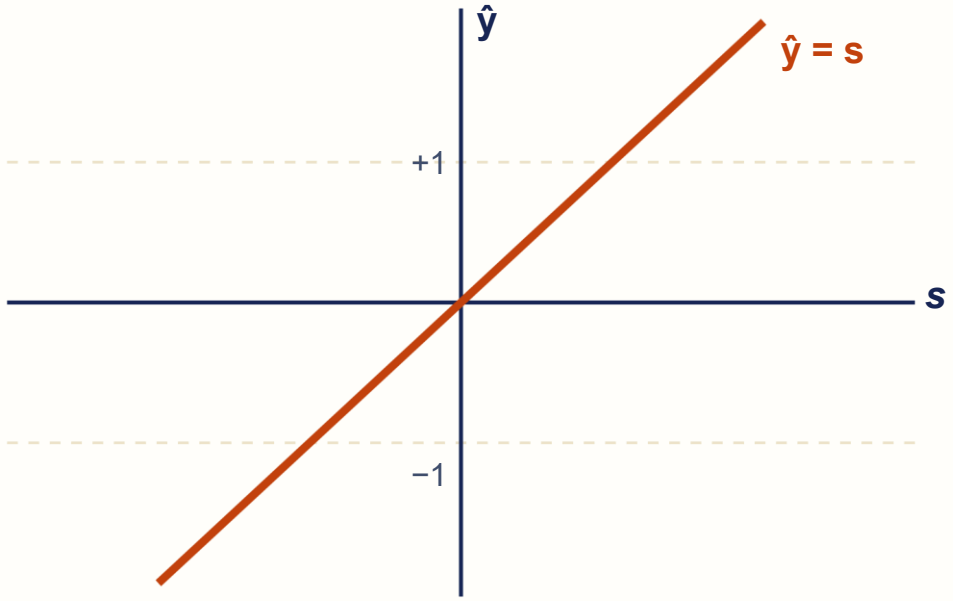
Perceptron = hồi quy tuyến tính + hàm dấu

Hồi quy tính điểm số, hàm dấu đưa ra nhãn.

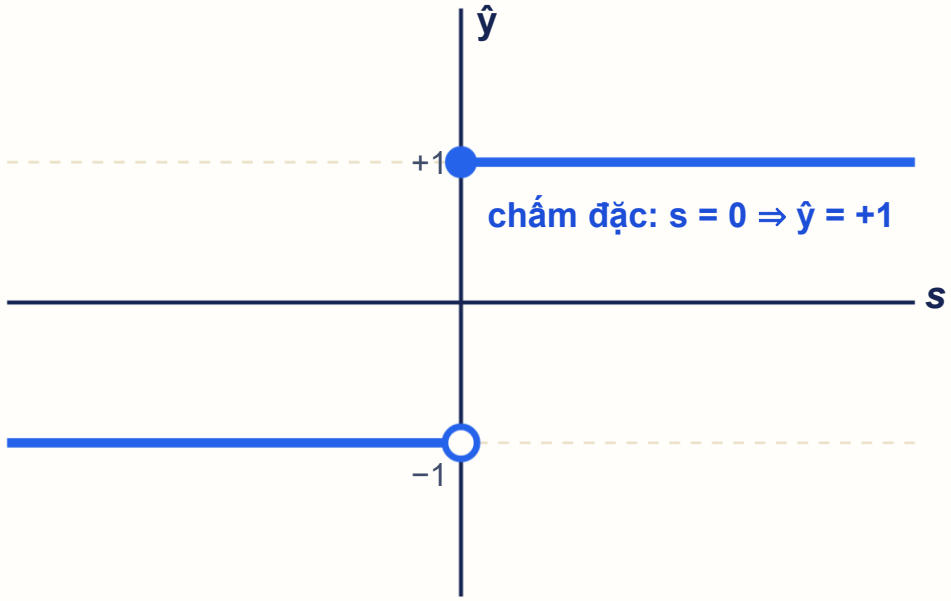
2.1 · SO SÁNH ĐẦU RA

Hồi quy giữ điểm số, Perceptron bẻ về ±1

Hồi quy tuyến tính: giữ nguyên s



Perceptron: chỉ ra −1 hoặc +1



2.1 · VÍ DỤ VÀ QUY ƯỚC TẠI S = 0

Dự đoán với **w** = [−2,5; 1; −1]^T

Sinh viên	x	s	ŷ
Học 5 giờ, vắng 2 buổi	[1; 5; 2]	−2,5 + 5 − 2 = 0,5	+1 (đậu)
Học 3 giờ, vắng 4 buổi	[1; 3; 4]	−2,5 + 3 − 4 = −3,5	−1 (rớt)
Học 4,5 giờ, vắng 2 buổi	[1; 4,5; 2]	−2,5 + 4,5 − 2 = 0	+1 (theo quy ước)

Quy ước: s = 0 thì ŷ = +1

Trong toán, sign(0) = 0. Nhưng nhãn chỉ có thể là −1 hoặc +1, nên ta chọn +1 và giữ nhất quán.

Hệ quả ở mục 3

Khởi tạo **w** = 0 thì mọi mẫu có s = 0, nên đều bị dự đoán +1.

2.1 · MỘT HỆ QUẢ QUAN TRỌNG

Hàm dấu không có đạo hàm hữu ích

Ở bài hồi quy, Gradient Descent cần đạo hàm của hàm mất mát theo tham số. Với Perceptron thì sao?

1

Hàm dấu là hàm hằng ở cả hai phía của s = 0.

2

Đạo hàm bằng 0 tại mọi s ≠ 0, không xác định tại s = 0.

3

Gradient của MSE bằng 0 hầu như mọi nơi: không có hướng để đi.

Cần một cách huấn luyện khác, bám sát việc đúng hay sai của từng mẫu.

Công cụ phù hợp là tích y·s đã gặp ở mục 1. Đây là nội dung mục 3.

2.2

Biểu diễn vector và ma trận của mô hình

Tính cho cả tập dữ liệu bằng một phép nhân ma trận.

2.2 · MỘT MẪU

Hai bước: nhân vector, rồi lấy dấu

$$s_i = \mathbf{x}_i \mathbf{w}$$

$$\hat{y}_i = \text{sign}(s_i)$$

Kiểm tra kích thước của phép nhân:

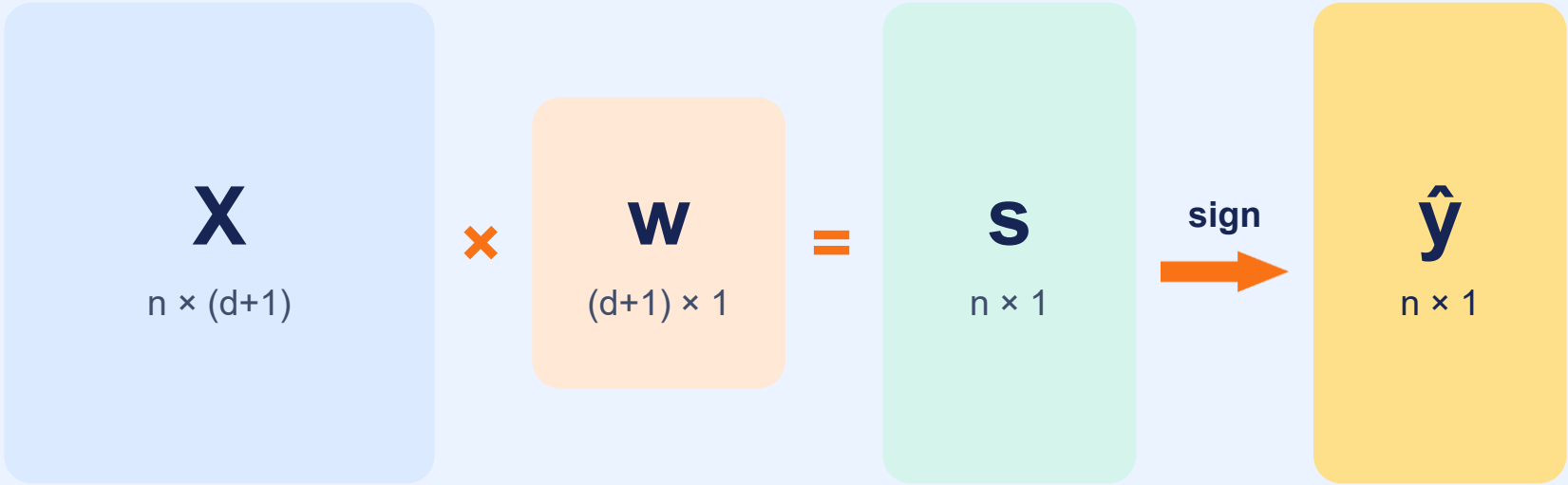
$\mathbf{x}_i$	·	$\mathbf{w}$	=	s_i
$1 \times (d+1)$		$(d+1) \times 1$		1×1

Vector hàng nhân vector cột cho ra một số: chính là điểm số s_i.

2.2 · TOÀN BỘ N MẪU CÙNG LÚC

Một phép nhân ma trận cho mọi điểm số

$$\mathbf{s} = \mathbf{X}\mathbf{w},\ \hat{\mathbf{y}} = \text{sign}(\mathbf{X}\mathbf{w})$$



Công thức ngắn gọn

Giống hồi quy, chỉ thêm hàm dấu

Lập trình không cần vòng lặp

2.2 · MARGIN

Mẫu đúng hay sai, và đúng đến mức nào

$$m_i = y_i\, s_i = y_i\, (\mathbf{x}_i \mathbf{w})$$

$m_i > 0$

Dự đoán đúng

m_i càng lớn, mẫu càng nằm sâu trong vùng đúng.

$m_i < 0$

Dự đoán sai

$|m_i|$ càng lớn, mẫu càng nằm sâu trong vùng sai.

$m_i = 0$

Nằm trên đường biên

Vì sign(0) = +1: đúng nếu y = +1, sai nếu y = −1.

$\mathbf{m} = \mathbf{y} \odot \mathbf{s}$ **Nhân từng phần tử của y với phần tử tương ứng của s (phép nhân Hadamard).**

2.2 · TỔNG HỢP KÍCH THƯỚC

Các đại lượng của mô hình

Đại lượng	Biểu thức	Kích thước
Ma trận đặc trưng	$\mathbf{X}$	$n \times (d+1)$
Vector tham số	$\mathbf{w}$	$(d+1) \times 1$
Vector điểm số	$\mathbf{s} = \mathbf{X}\mathbf{w}$	$n \times 1$
Vector nhãn dự đoán	$\hat{\mathbf{y}} = \text{sign}(\mathbf{s})$	$n \times 1$
Vector nhãn thực	$\mathbf{y}$	$n \times 1$
Vector margin	$\mathbf{m} = \mathbf{y} \odot \mathbf{s}$	$n \times 1$

2.2 · VÍ DỤ SỐ VỚI DỮ LIỆU SINH VIÊN

Margin cho biết mẫu nào sai và sai nặng đến đâu

Bộ tốt: **w** = $[-2,5; 1; -1]^T$

SV	s	ŷ	m	Kết quả
1	−4,5	−1	4,5	Đúng
2	−2,5	−1	2,5	Đúng
3	−3,5	−1	3,5	Đúng
4	2,5	+1	2,5	Đúng
5	3,5	+1	3,5	Đúng
6	6,5	+1	6,5	Đúng

ŷ trùng y và mọi margin đều dương.

Bộ chưa tốt: **w** = $[-2,5; 1; 0]^T$

SV	s	ŷ	m	Kết quả
1	−0,5	−1	0,5	Đúng
2	0,5	+1	−0,5	Sai: báo nhầm
3	1,5	+1	−1,5	Sai: báo nhầm
4	3,5	+1	3,5	Đúng
5	5,5	+1	5,5	Đúng
6	6,5	+1	6,5	Đúng

Mô hình bỏ qua số buổi vắng. Hai margin âm ở SV 2 và 3.

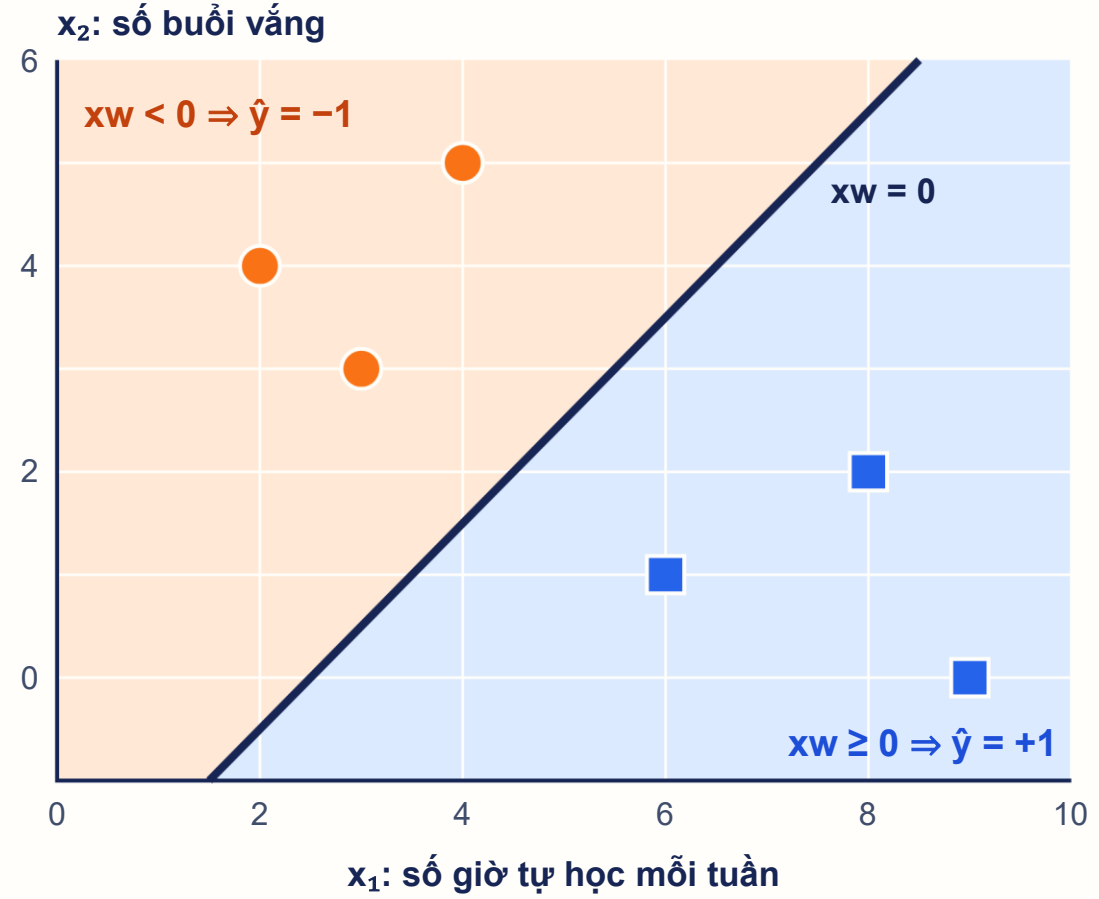
2.3

Đường biên quyết định và không gian đặc trưng

Tham số điều khiển việc chia như thế nào.

2.3 · ĐƯỜNG BIÊN QUYẾT ĐỊNH

Tập các điểm có điểm số bằng đúng 0



$$\{ x : xw = 0 \}$$

$$w_0 + w_1 x_1 + \ldots + w_d x_d = 0$$

Nửa không gian $xw \geq 0$

Gồm cả biên: dự đoán $\hat{y} = +1$.

Nửa không gian $xw < 0$

Dự đoán $\hat{y} = -1$.

Đường biên quyết định (decision boundary) chia không gian đặc trưng thành hai nửa không gian.

2.3 · HÌNH DẠNG CỦA ĐƯỜNG BIÊN

Phụ thuộc vào số đặc trưng d

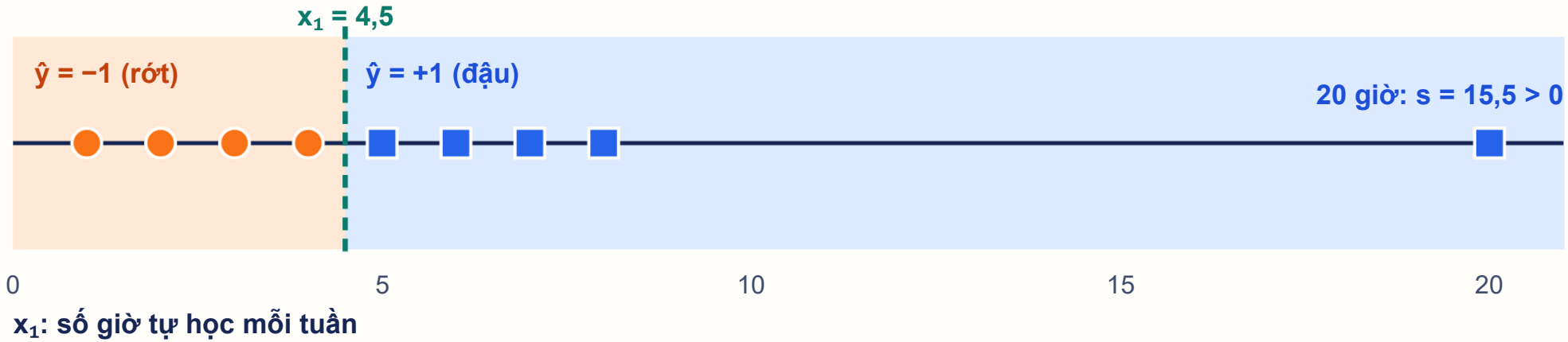
d	Đường biên là	Phương trình
1	Một điểm	$w_0 + w_1x_1 = 0$
2	Một đường thẳng	$w_0 + w_1x_1 + w_2x_2 = 0$
3	Một mặt phẳng	$w_0 + w_1x_1 + w_2x_2 + w_3x_3 = 0$
> 3	Một siêu phẳng	$\mathbf{xw} = 0$

Dù số chiều là bao nhiêu, đường biên luôn là một đối tượng "phẳng".

Vì vậy Perceptron là một bộ phân loại tuyến tính (linear classifier).

2.3 · TRƯỜNG HỢP D = 1

Đường biên là một điểm: $x_1 = -w_0 / w_1$



Hồi quy + ngưỡng (mục 1.2)

Thêm SV học 20 giờ làm ngưỡng dịch từ 4,5 sang 5,27.

Perceptron, w = $[-4{,}5;\ 1]^T$

Biên vẫn ở 4,5. Mẫu “rất đúng” không làm lệch biên, vì mô hình chỉ xét điểm nằm về phía nào.

2.3 · TRƯỜNG HỢP D = 2

Rút x_2 theo x_1 để vẽ đường thẳng

Tổng quát ($w_2 \neq 0$)

$$x_2 = -w_0/w_2 - (w_1/w_2)\cdot x_1$$

Với **w** = $[-2{,}5;\ 1;\ -1]^T$

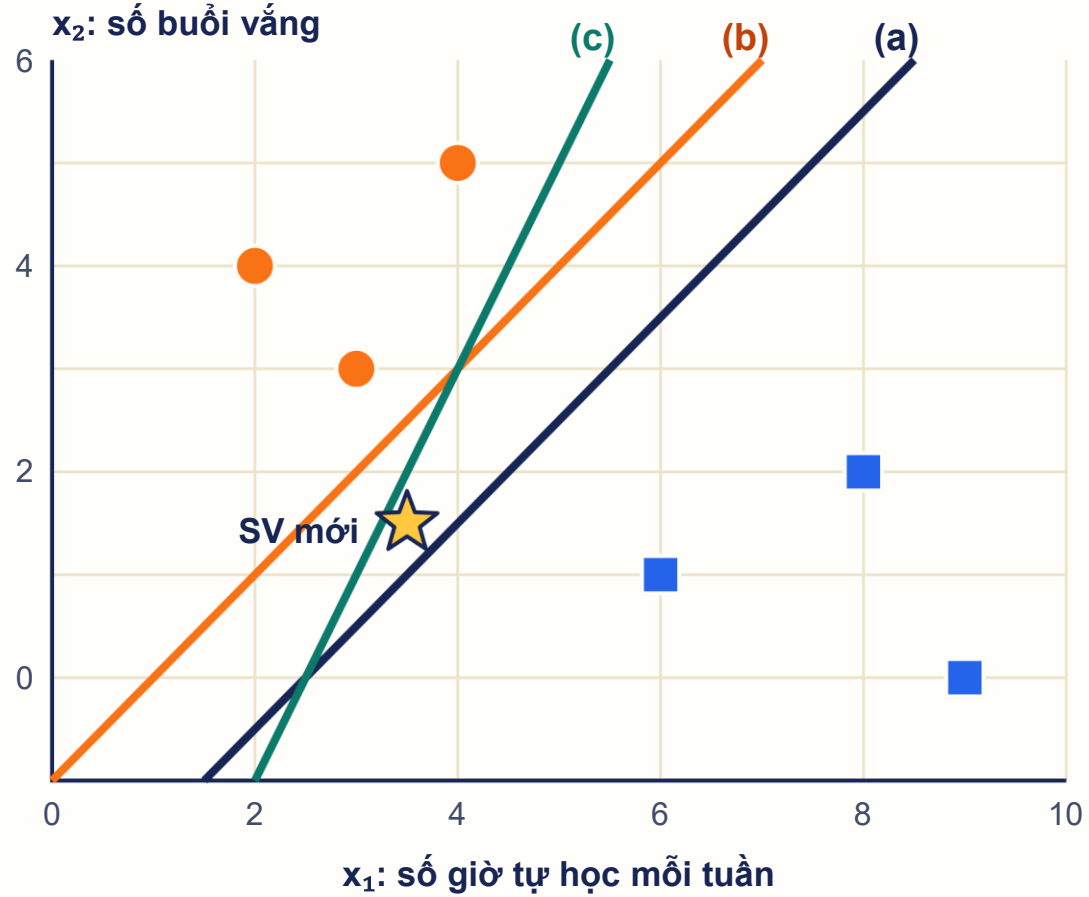
$$x_2 = -(-2{,}5)/(-1) - (1/(-1))\cdot x_1$$

$$x_2 = x_1 - 2{,}5$$

Đúng là đường thẳng đã vẽ ở mục 1: $x_1 - x_2 - 2{,}5 = 0$.

2.3 · VAI TRÒ CỦA CÁC THAM SỐ

Đổi w_0 thì tịnh tiến, đổi trọng số thì xoay



(a) Ban đầu: w = $[-2{,}5;\ 1;\ -1]^T$

Đường biên $x_2 = x_1 - 2{,}5$.

(b) Đổi w_0: $-2{,}5 \rightarrow -1$

Đường biên $x_2 = x_1 - 1$: **tịnh tiến** song song.

(c) Đổi w_2: $-1 \rightarrow -0{,}5$

Đường biên $x_2 = 2x_1 - 5$: **xoay** quanh điểm (2,5; 0).

Cả ba đường đều phân loại đúng 6 sinh viên ban đầu.

2.3 · DỮ LIỆU MỚI

Cùng đúng trên dữ liệu cũ, khác nhau trên dữ liệu mới

Sinh viên mới: học 3,5 giờ, vắng 1,5 buổi, x = [1; 3,5; 1,5].

Bộ tham số w	s	Nhãn dự đoán
(a) $[-2,5; 1; -1]^T$, ban đầu	−0,5	−1 (rớt)
(b) $[-1; 1; -1]^T$, đổi w_0	1	+1 (đậu)
(c) $[-2,5; 1; -0,5]^T$, đổi w_2	0,25	+1 (đậu)

Việc chọn w rất quan trọng.

Nhiều bộ tham số cùng đúng trên dữ liệu huấn luyện vẫn có thể dự đoán khác nhau trên dữ liệu mới. Ta sẽ quay lại ở mục 4.

2.3 · MỘT TÍNH CHẤT HỮU ÍCH

Nhân w với hằng số dương không đổi bộ phân loại

$$\text{sign}(\mathbf{x}(c\mathbf{w})) = \text{sign}(c \cdot \mathbf{x}\mathbf{w}) = \text{sign}(\mathbf{x}\mathbf{w}),\ c > 0$$

Ban đầu

$\mathbf{w} = [-2{,}5;\ 1;\ -1]^T$

× 2

Sau khi nhân

$2\mathbf{w} = [-5;\ 2;\ -2]^T$

Không đổi

Đường biên và nhãn dự đoán: nhân s với số dương không đổi dấu.

Có đổi

Margin m_i: được nhân với c.

Điều quan trọng

Là hướng của **w**, không phải độ lớn tuyệt đối.

2.3 · GIỚI HẠN CỦA MÔ HÌNH

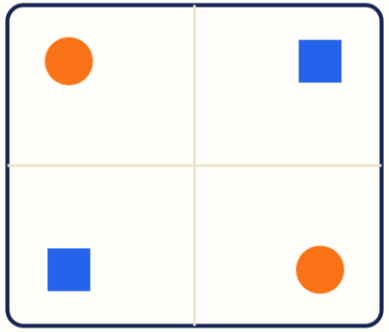
Đường biên luôn là đường phẳng

Perceptron chỉ phân loại hoàn hảo được dữ liệu **phân tách tuyến tính được**:

$$y_i \, \mathbf{x}_i \mathbf{w} > 0, \text{ mọi } i$$

Khi hai lớp đan xen nhau (như bài toán XOR ở mục 4), không có đường thẳng nào làm đúng cả: Perceptron chắc chắn mắc sai sót.

Đây là động lực để phát triển các mô hình mạnh hơn, như mạng nơ-ron nhiều lớp.



Ví dụ: XOR

Không đường thẳng nào tách được hai nhóm.

TÓM TẮT MỤC 2

Năm ý cần nhớ

1. **Điểm số** $s = \mathbf{xw} = w_0 + w_1x_1 + \ldots + w_dx_d$, rồi **hàm dấu** cho nhãn: $\hat{y} = \text{sign}(\mathbf{xw})$.

2. **Perceptron = hồi quy tuyến tính + hàm dấu.** Hàm dấu có đạo hàm bằng 0 hầu như mọi nơi nên cần cách huấn luyện khác Gradient Descent trên MSE.

3. Cho cả tập dữ liệu: $\hat{\mathbf{y}} = \text{sign}(\mathbf{Xw})$. **Margin** $m_i = y_i s_i$: dương là đúng, âm là sai, độ lớn là mức đúng sai.

4. **Đường biên** là siêu phẳng $\mathbf{xw} = 0$. Đổi w_0 thì tịnh tiến, đổi trọng số thì xoay. Nhân w với số dương không đổi bộ phân loại.

5. Chỉ phân loại hoàn hảo được dữ liệu **phân tách tuyến tính được**.

KIỂM TRA NHANH

Năm câu hỏi cho sinh viên

1. Với $\mathbf{w} = [-2{,}5;\ 1;\ -1]^T$ và $x = [1;\ 6;\ 3]$, tính s và ŷ.

2. Cùng **w**, với **x** = [1; 4; 1,5], tính s và ŷ. Kết quả phụ thuộc vào quy ước nào?

3. Cho $\mathbf{w} = [-6;\ 2;\ 3]^T$. Viết phương trình đường biên dạng x_2 theo x_1. Gốc tọa độ (0, 0) nằm ở phía được dự đoán nhãn nào?

4. Với $\mathbf{w} = [-2{,}5;\ 0;\ -1]^T$ trên 6 sinh viên, tính **ŷ** và **m**. Có bao nhiêu mẫu bị phân loại sai?

5. Vì sao không thể huấn luyện Perceptron bằng cách áp dụng trực tiếp Gradient Descent lên MSE giữa y và ŷ = sign(**xw**)?

GỢI Ý ĐÁP ÁN

Đáp án kiểm tra nhanh

1 $s = -2{,}5 + 6 - 3 = 0{,}5 \geq 0$ nên $\hat{y} = +1$.

2 $s = -2{,}5 + 4 - 1{,}5 = 0$: điểm nằm trên đường biên, nên ŷ phụ thuộc quy ước tại $s = 0$. Theo bài giảng, $\hat{y} = +1$.

3 $x_2 = 2 - (2/3)x_1$. Tại (0, 0), $s = -6 < 0$ nên gốc tọa độ thuộc phía $\hat{y} = -1$.

4 $s = -2{,}5 - x_2 = [-6{,}5; -5{,}5; -7{,}5; -3{,}5; -4{,}5; -2{,}5]^T$ nên ŷ toàn −1. **m** = $[6{,}5; 5{,}5; 7{,}5; -3{,}5; -4{,}5; -2{,}5]^T$: có 3 mẫu sai (SV 4, 5, 6), đều là lỗi bỏ sót.

5 Hàm dấu là hàm hằng ở hai phía của $s = 0$ nên đạo hàm bằng 0 tại mọi $s \neq 0$. Gradient theo w bằng 0 hầu như mọi nơi, Gradient Descent không có hướng để cập nhật.

HỌC MÁY · BÀI 2 · PERCEPTRON

Huấn luyện mô hình Perceptron

Mục 3 · Từ mẫu sai đến tham số w tốt



MỤC TIÊU CỦA MỤC 3

Ba câu hỏi cần trả lời

1

Làm thế nào đo mức độ sai của một bộ tham số w?

Hàm mất mát: mục 3.1 và 3.2.

2

Khi gặp một mẫu sai, nên chỉnh w theo hướng nào?

Quy tắc cập nhật: mục 3.3.

3

Lặp lại việc chỉnh sửa ra sao, và khi nào dừng?

Thuật toán, ví dụ tính tay và tốc độ học: mục 3.4 đến 3.6.

3.1

Khung huấn luyện: còn thiếu gì?

Đặt Perceptron cạnh hồi quy tuyến tính.

3.1 · NHẮC LẠI KHUNG HUẤN LUYỆN

Thành phần nào đã có, thành phần nào còn thiếu?

Thành phần	Hồi quy tuyến tính	Perceptron
Mô hình	$\hat{y} = \mathbf{xw}$	$\hat{y} = \text{sign}(\mathbf{xw})$ (đã có, mục 2)
Hàm mất mát	MSE	Mất mát Perceptron (mục 3.2)
Thuật toán tối ưu	Gradient Descent	Gradient Descent từng mẫu (3.3, 3.4)

Mục tiêu vẫn như cũ: tìm w làm hàm mất mát nhỏ nhất.

3.1 · HÀM MẤT MÁT NÊN LÀ GÌ?

Số lỗi 0–1 phản ánh đúng, nhưng không có độ dốc

Điều ta quan tâm

Số mẫu bị phân loại sai. Mất mát 0–1: bằng 1 nếu sai, 0 nếu đúng.

Vấn đề

Hàm hằng từng khúc: gradient bằng 0 hầu như mọi nơi, không biết chỉnh w theo hướng nào.

Cần hàm mất mát vẫn phản ánh đúng hay sai, nhưng có độ dốc.

Công cụ phù hợp: margin $m_i = y_i s_i$ đã gặp ở mục 2.

3.2

Hàm mất mát của Perceptron

Chỉ phạt mẫu sai, sai càng nặng phạt càng nhiều.

3.2 · Ý TƯỞNG

Dùng margin $m_i = y_i \cdot s_i$ để phạt

$m > 0$

Mẫu đúng

Không cần phạt.

$m < 0$

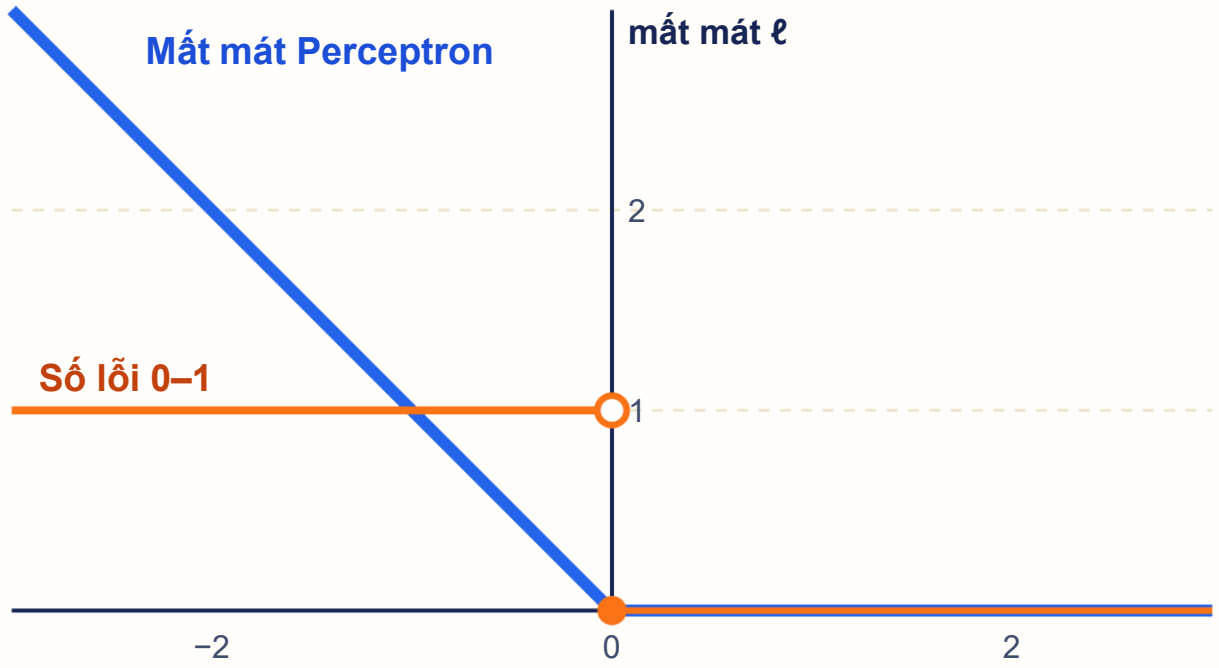
Mẫu sai

Phạt. |m| càng lớn (mẫu càng sâu trong vùng sai) thì phạt càng nặng.

Mất mát tỉ lệ với mức độ sai, thay vì chỉ đếm “sai hay đúng”.

3.2 · SO SÁNH THEO MARGIN

Số lỗi 0–1 và mất mát Perceptron



Số lỗi 0–1

Mẫu sai luôn bằng 1, bất kể sai ít hay nhiều. Phần dốc bằng 0.

Mất mát Perceptron

Tăng dần theo mức độ sai và có độ dốc khác 0, nên dùng được đạo hàm.

Cả hai đều bằng 0 khi mẫu đúng.

3.2 · ĐỊNH NGHĨA

Mất mát một mẫu và mất mát tổng

Mẫu thứ i

$$\ell_i(\mathbf{w}) = \max(0, -y_i\, \mathbf{x}_i\mathbf{w}) = \max(0, -m_i)$$

= 0 nếu $m_i \geq 0$; = $-m_i$ nếu $m_i < 0$

Toàn bộ dữ liệu

$$J(\mathbf{w}) = \Sigma_i \max(0, -y_i\, \mathbf{x}_i\mathbf{w}) = \Sigma_{\{i \in M(\mathbf{w})\}} (-y_i\, \mathbf{x}_i\mathbf{w})$$

$M(\mathbf{w}) = \{ i : y_i\, \mathbf{x}_i\mathbf{w} \leq 0 \}$ là tập các mẫu bị sai (hoặc chưa đúng chắc chắn).

J(w) chỉ cộng dồn phần "lỗi" của những mẫu sai.

3.2 · TÍNH CHẤT CỦA J(W)

Ba tính chất và một cái bẫy

1

$J(w) \geq 0$ với mọi w.

2

$J(w) = 0$ khi mọi mẫu có $m_i \geq 0$: không còn mẫu nào sai phía.

3

Mỗi mẫu sai đóng góp $|s_i|$: J tỉ lệ với tổng khoảng cách từ các mẫu sai đến đường biên.

Lưu ý: nghiệm tầm thường

Với $\mathbf{w} = \mathbf{0}$, mọi margin bằng 0 nên $J(\mathbf{0}) = 0$, nhưng mô hình chưa phân biệt được gì.

Vì vậy ta coi mẫu có $m_i \leq 0$ (kể cả $m_i = 0$) là mẫu **cần cập nhật**. Đó là lý do $M(\mathbf{w})$ dùng dấu ≤.

3.2 · VÍ DỤ SỐ VỚI DỮ LIỆU SINH VIÊN

Bộ tham số càng tệ thì J càng lớn

w	Vector margin m	Mẫu sai	J(w)
$[-2,5; 1; -1]^T$	[4,5; 2,5; 3,5; 2,5; 3,5; 6,5]	Không có	0
$[-2,5; 1; 0]^T$	[0,5; −0,5; −1,5; 3,5; 5,5; 6,5]	SV 2, 3	0,5 + 1,5 = 2
$[-2,5; 0; -1]^T$	[6,5; 5,5; 7,5; −3,5; −4,5; −2,5]	SV 4, 5, 6	3,5 + 4,5 + 2,5 = 10,5

J = 0 ứng với bộ tham số phân loại đúng toàn bộ.

Đúng là hành vi ta mong muốn ở một hàm mất mát.

3.3

Quy tắc cập nhật khi phân loại sai

Gradient của mất mát một mẫu.

3.3 · GRADIENT CỦA $\ell_i(W)$

Hai trường hợp: mẫu đúng và mẫu sai

Mẫu đúng ($m > 0$)

$\ell_i = 0$ là hằng số nên gradient là vector 0.

$$\nabla \ell_i = \mathbf{0}$$

Không ảnh hưởng đến việc cập nhật.

Mẫu sai ($m \le 0$)

$\ell_i = -y_i \mathbf{x}_i \mathbf{w}$ là hàm tuyến tính của $\mathbf{w}$.

$$\nabla \ell_i = -y_i \mathbf{x}_i^T$$

Dùng $\mathbf{x}_i^T$ (vector cột) để cùng kích thước $(d+1) \times 1$ với $\mathbf{w}$.

Tại $m_i = 0$, hàm $\max(0, -m)$ có "góc nhọn" nên đạo hàm không xác định. Ta lấy $-y_i x_i^T$ làm hướng cập nhật.

3.3 · QUY TẮC CẬP NHẬT PERCEPTRON

Cộng hoặc trừ vector đặc trưng vào w

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \mathbf{y_i} \cdot \mathbf{x_i}^T \text{ nếu } y_i\, \mathbf{x_i w} \le 0$$

$$\mathbf{w} \leftarrow \mathbf{w} \text{ nếu } y_i\, \mathbf{x_i w} > 0$$

$\eta > 0$ là tốc độ học. Từng thành phần: $w_0 \leftarrow w_0 + \eta \cdot y_i$, $w_j \leftarrow w_j + \eta \cdot y_i \cdot x_{ij}$.

$y_i = +1$: bỏ sót

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \mathbf{x_i}^T$$

Cộng vector đặc trưng vào **w**: kéo **w** lại gần $\mathbf{x_i}$.

$y_i = -1$: báo nhầm

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \mathbf{x_i}^T$$

Trừ vector đặc trưng khỏi **w**: đẩy **w** ra xa $\mathbf{x_i}$.

3.3 · VÍ DỤ MỘT LẦN CẬP NHẬT

$\mathbf{w} = \mathbf{0}$, $\eta = 1$, sinh viên 1

$\mathbf{x_1} = [1, 2, 4]$, $y_1 = -1$

$m_1 = (-1)\cdot 0 = 0 \le 0$

Cần cập nhật.

$y_1 = -1$ nên **trừ** $\mathbf{x_1}^T$ khỏi $\mathbf{w}$:

$$\mathbf{w} = [0; 0; 0] + (-1)\cdot[1; 2; 4]$$

$$\mathbf{w} = [-1; -2; -4]^T$$

Sinh viên 1 thuộc lớp rớt nên w bị "đẩy ra xa" $\mathbf{x_1}$: trọng số của giờ học và buổi vắng đều giảm.

3.3 · VÌ SAO QUY TẮC NÀY HỢP LÝ?

Margin của mẫu vừa cập nhật luôn tăng

$$m' = y_i \mathbf{x}_i (\mathbf{w} + \eta\, y_i \mathbf{x}_i^T) = y_i \mathbf{x}_i \mathbf{w} + \eta\, y_i^2 \mathbf{x}_i \mathbf{x}_i^T$$

$$m' = m + \eta \|\mathbf{x}_i\|^2$$

Dùng $y_i^2 = 1$ và $\mathbf{x}_i \mathbf{x}_i^T = \|\mathbf{x}_i\|^2 > 0$ (phần tử đầu của $\mathbf{x}_i$ luôn bằng 1).

Ví dụ

$\|\mathbf{x}_1\|^2 = 1 + 4 + 16 = 21$: margin SV 1 tăng từ 0 lên 21, khớp $s_1 = -21$ sau cập nhật.

Cảnh báo

Một lần cập nhật chỉ cải thiện mẫu đang xét, có thể làm xấu các mẫu khác. Vì vậy phải lặp lại nhiều lần.

3.3 · SO SÁNH VỚI BÀI HỒI QUY

Cùng dạng $w \leftarrow w - \eta \cdot (\text{gradient})$, khác hai điểm

HỒI QUY TUYẾN TÍNH

Mọi mẫu đều đóng góp vào gradient, độ lớn tỉ lệ với sai số liên tục.

Thường duyệt toàn bộ dữ liệu rồi mới cập nhật một lần.

PERCEPTRON

Chỉ mẫu sai mới đóng góp, mức đóng góp luôn là $\pm \eta \cdot x_i^T$.

Cập nhật **ngay sau mỗi mẫu**.

Cập nhật theo từng mẫu gọi là Gradient Descent ngẫu nhiên (SGD).

Stochastic gradient descent: mỗi lần chỉ dùng một mẫu.

3.3 · DẠNG MA TRẬN (THAM KHẢO)

Gom tất cả mẫu sai trong một lần

Đặt $\mathbf{u} \in \mathbb{R}^n$ với $u_i = y_i$ nếu $i \in M(\mathbf{w})$, ngược lại $u_i = 0$.

$$\nabla J(\mathbf{w}) = -\Sigma\, y_i\, \mathbf{x_i}^T = -\mathbf{X}^T\mathbf{u}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \mathbf{X}^T\mathbf{u}$$

$\mathbf{X}^T$	×	$\mathbf{u}$	=	$\mathbf{X}^T\mathbf{u}$
(d+1) × n		n × 1		(d+1) × 1

Kích thước khớp với w. Bài giảng dùng dạng từng mẫu vì đó là Perceptron kinh điển và dễ tính tay hơn.

3.4

Thuật toán Perceptron

Từng bước, từ khởi tạo đến điều kiện dừng.

3.4 · BA KHÁI NIỆM CẦN NẮM

Epoch, cập nhật tức thời, thứ tự duyệt

Epoch

Một lượt duyệt qua **toàn bộ n mẫu** của tập dữ liệu.

Cập nhật tức thời

Gặp mẫu sai thì sửa **w ngay**; mẫu sau dùng w mới.

Thứ tự duyệt

Có thể giữ cố định 1→n hoặc **xáo trộn** mỗi epoch.

Trong ví dụ của bài, ta duyệt cố định theo thứ tự 1 → 6.

Cố định giúp tính tay có thể lặp lại; xáo trộn thường dùng khi lập trình.

3.4 · THUẬT TOÁN PERCEPTRON

Giả mã trong chín dòng

1 Khởi tạo w (thường w = 0), chọn η > 0, chọn T_max
2 for epoch = 1, 2, …, T_max:
3 c ← 0 (đếm số lần cập nhật)
4 for i = 1, …, n:
5 if $y_i \cdot x_i w \le 0$:
6 $w \leftarrow w + \eta \cdot y_i \cdot x_i^T$
7 c ← c + 1
8 if c = 0: dừng (đã phân loại đúng hết)
9 trả về w

Đầu vào

Dữ liệu (X, y), η, T_max.

Đầu ra

Vector w cho ŷ = sign(xw).

3.4 · KHI NÀO DỪNG?

Hai điều kiện dừng

c = 0: hội tụ

Một epoch trọn vẹn không có cập nhật nào: mọi mẫu có $m > 0$, w phân loại đúng toàn bộ.

Đạt T_max: van an toàn

Giới hạn số epoch để thuật toán không chạy mãi khi dữ liệu không phân tách tuyến tính được.

Epoch cuối cùng chỉ để xác nhận: nó không thay đổi w.

Vì vậy số epoch cần chạy luôn bằng số epoch có cập nhật cộng thêm một.

3.5

Ví dụ tính tay một vòng huấn luyện

Sáu sinh viên, $w = 0$, $\eta = 1$.

3.5 · DỮ LIỆU VÀ THIẾT LẬP

Sáu sinh viên: $\mathbf{x}$ = [1, giờ học, số buổi vắng]

Sinh viên	x_i	y_i
1	[1, 2, 4]	−1 · rớt
2	[1, 3, 3]	−1 · rớt
3	[1, 4, 5]	−1 · rớt
4	[1, 6, 1]	+1 · đậu
5	[1, 8, 2]	+1 · đậu
6	[1, 9, 0]	+1 · đậu

Khởi tạo

$\mathbf{w} = [0; 0; 0]^T$

Tốc độ học

$\eta = 1$

Thứ tự

Duyệt 1 → 6, cập nhật tức thời

3.5 · EPOCH 1

Duyệt sáu mẫu: hai lần cập nhật

Mẫu	s_i	m_i	Quyết định	w sau
1	0	0	Sai ($m \le 0$) → cập nhật	[−1; −2; −4]
2	−19	19	Đúng, giữ nguyên	[−1; −2; −4]
3	−29	29	Đúng, giữ nguyên	[−1; −2; −4]
4	−17	−17	Sai → cập nhật	[0; 4; −3]
5	26	26	Đúng, giữ nguyên	[0; 4; −3]
6	36	36	Đúng, giữ nguyên	[0; 4; −3]

Kiểm tra lại toàn bộ với w = $[0; 4; -3]^T$: **s** = [−4, 3, 1, 21, 26, 36]

m = [4, −3, −1, 21, 26, 36], sinh viên 2 và 3 vẫn bị sai nên phải chạy tiếp.

3.5 · CÁC EPOCH TIẾP THEO

Chỉ liệt kê những lần có cập nhật

Lần	Epoch	Mẫu	s trước	m	w sau
1	1	1	0	0	[−1; −2; −4]
2	1	4	−17	−17	[0; 4; −3]
3	2	2	3	−3	[−1; 1; −6]
4	2	4	−1	−1	[0; 7; −5]
5	3	2	6	−6	[−1; 4; −8]

Tổng cộng 5 lần cập nhật: 2 ở epoch 1, 2 ở epoch 2, 1 ở epoch 3.

Epoch 4 duyệt cả sáu mẫu mà không cập nhật lần nào.

3.5 · EPOCH 2 VÀ 3

Hai epoch tiếp theo

Epoch 2 (c = 2)

Mẫu 1 đúng (m = 4). **Mẫu 2 sai** (m = −3): **w** = $[-1; 1; -6]^T$. Mẫu 3 đúng. **Mẫu 4 sai** (m = −1): **w** = $[0; 7; -5]^T$. Mẫu 5, 6 đúng.

Epoch 3 (c = 1)

Mẫu 1 đúng (m = 6). **Mẫu 2 sai** (m = −6): **w** = $[-1; 4; -8]^T$. Mẫu 3, 4, 5, 6 đều đúng (m = 25, 15, 15, 35).

Cuối epoch 3, w = $[-1; 4; -8]^T$ nhưng vì c = 1 nên ta chưa dừng được.

Phải chạy thêm một epoch để xác nhận.

3.5 · EPOCH 4 VÀ KẾT QUẢ

Epoch 4: c = 0, thuật toán dừng

Kết quả huấn luyện

$\mathbf{w} = [-1; 4; -8]^T$

$\mathbf{s} = [-25, -13, -25, 15, 15, 35]$

$\hat{\mathbf{y}} = [-1, -1, -1, +1, +1, +1]$

$\mathbf{m} = [25, 13, 25, 15, 15, 35]$

Đường biên quyết định

$-1 + 4x_1 - 8x_2 = 0$

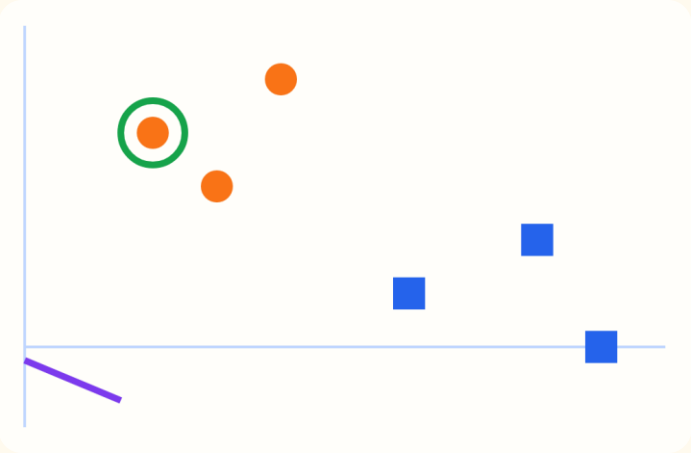
$\Leftrightarrow x_2 = x_1/2 - 1/8$

Mọi margin đều dương: J(w) = 0, phân loại đúng cả sáu sinh viên.

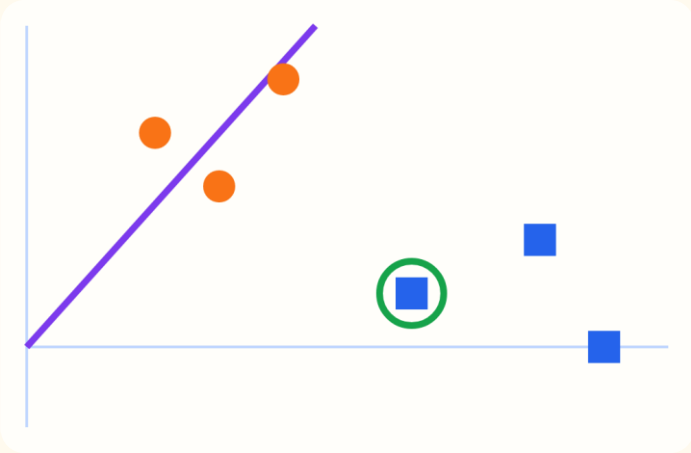
Dự đoán sinh viên mới (học 5 giờ, vắng 1,5 buổi): $s = 7 \geq 0$, nên $\hat{y} = +1$ (đậu).

3.5 · NHÌN BẰNG HÌNH

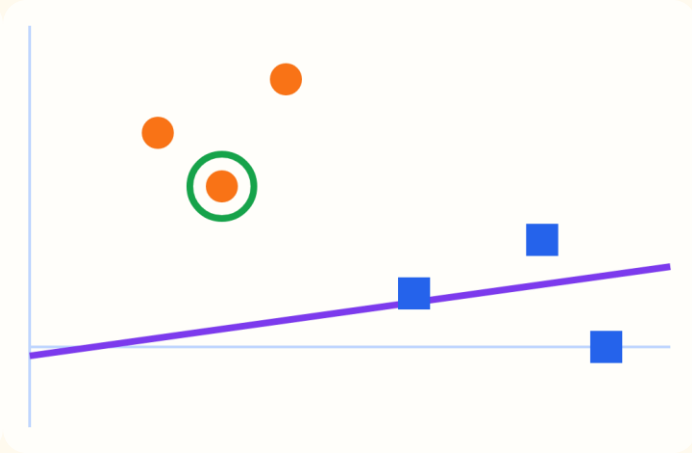
Đường biên “xoay” dần sau mỗi lần cập nhật



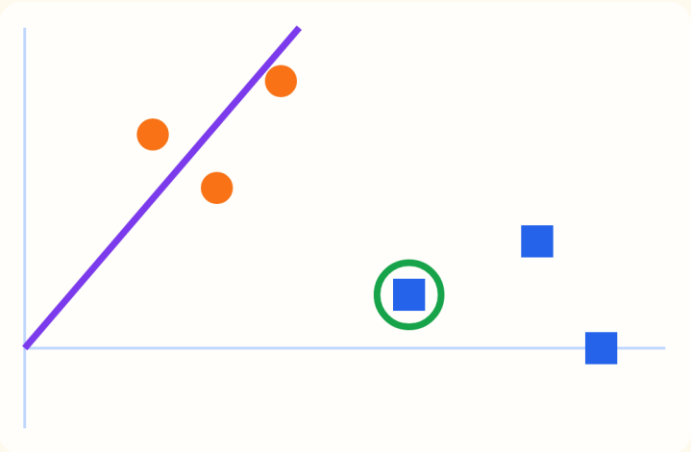
Lần 1 (SV 1): $\mathbf{w} = [-1; -2; -4]^T$



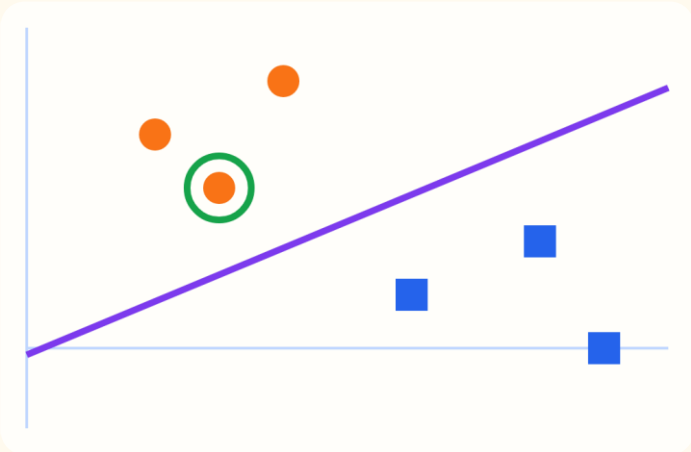
Lần 2 (SV 4): $\mathbf{w} = [0; 4; -3]^T$



Lần 3 (SV 2): $\mathbf{w} = [-1; 1; -6]^T$



Lần 4 (SV 4): $\mathbf{w} = [0; 7; -5]^T$



Lần 5 (SV 2): $\mathbf{w} = [-1; 4; -8]^T$

Vòng xanh: mẫu vừa được cập nhật.

Hình vuông xanh = đậu, tròn cam = rớt. Đường tím là đường biên.

3.5 · NHẬN XÉT

Ba điều rút ra từ ví dụ

Kéo co

Mẫu 2 và mẫu 4 bị sửa nhiều lần: sửa mẫu này có thể làm hỏng mẫu kia.

Vô số nghiệm

Nhiều w khác cũng phân loại đúng. Kết quả phụ thuộc thứ tự duyệt và khởi tạo.

Chỉ dừng khi đúng hết

Thuật toán chỉ dừng khi mọi mẫu đúng; nếu dữ liệu không tách được, nó chạy đến T_max.

Perceptron tìm một đường biên đúng, không phải đường biên tốt nhất.

Câu hỏi “đường nào tốt nhất?” dẫn đến các mô hình như SVM.

3.6

Vai trò của tốc độ học η

Ở hồi quy, η rất nhạy. Ở Perceptron thì sao?

3.6 · KHỞI TẠO W = 0

η chỉ co giãn w, không đổi bộ phân loại

η	Số lần cập nhật	w cuối cùng
0,1	5	$[-0{,}1;\ 0{,}4;\ -0{,}8]^T$
1	5	$[-1;\ 4;\ -8]^T$
10	5	$[-10;\ 40;\ -80]^T$

Cùng 5 lần cập nhật, cùng thứ tự mẫu: $\mathbf{w}^{(\eta)} = \eta \cdot \mathbf{w}^{(1)}$.

Ba bộ tham số cho cùng đường biên và cùng nhãn dự đoán.

3.6 · VÌ SAO?

Chứng minh bằng quy nạp

Giả sử $\mathbf{w}_t^{(\eta)} = \eta \cdot \mathbf{w}_t^{(1)}$

$$\mathbf{w}_{\{t+1\}}^{(\eta)} = \eta \cdot \mathbf{w}_t^{(1)} + \eta \cdot y_i \cdot \mathbf{x}_i^T = \eta \cdot (\mathbf{w}_t^{(1)} + y_i \cdot \mathbf{x}_i^T) = \eta \cdot \mathbf{w}_{\{t+1\}}^{(1)}$$

Cơ sở: $w_0^{(\eta)} = 0 = \eta \cdot 0$.

Lý do 1

Nhân **w** với số dương không đổi dấu của s, nên điều kiện m ≤ 0 giữ nguyên.

Lý do 2

Hai lần chạy chọn đúng cùng một mẫu để cập nhật ở mỗi bước.

3.6 · KHỞI TẠO W ≠ 0

Lúc này η có ảnh hưởng

Ví dụ: $\mathbf{w}^{(0)} = [1; 1; 1]^T$. η thể hiện tỉ lệ giữa bước cập nhật và độ lớn của $\mathbf{w}^{(0)}$.

η	Số lần cập nhật	w cuối cùng
0,1	6	$[0{,}6; 0{,}2; -0{,}8]^T$
1	3	$[0; 3; -6]^T$
10	5	$[-9; 41; -79]^T$

Số lần cập nhật và w cuối đều khác nhau, nhưng cả ba đều phân loại đúng toàn bộ dữ liệu.

3.6 · KẾT LUẬN

Perceptron ít nhạy với η hơn hồi quy

Hội tụ

Nếu dữ liệu tách tuyến tính được, thuật toán dừng sau hữu hạn bước với **mọi** $\eta > 0$.

Thực hành

Với $\mathbf{w} = \mathbf{0}$, η chỉ co giãn w. Thường chọn $\boldsymbol{\eta} = \mathbf{1}$.

Chỉ đúng với Perceptron

Với hàm mất mát trơn (hồi quy tuyến tính, Logistic), gradient phụ thuộc **w** nên η rất quan trọng.

Đừng mang kết luận này sang các mô hình khác.

Hàm mất mát trơn cần tốc độ học được chọn cẩn thận.

TÓM TẮT MỤC 3

Năm ý cần nhớ

1. Huấn luyện = tìm $\mathbf{w}$ để mọi margin $m_i = y_i \mathbf{x}_i \mathbf{w}$ đều dương. Số lỗi 0–1 không dùng được với gradient nên cần mất mát khác.

2. Mất mát Perceptron: $\ell_i = \max(0, -m_i)$; $J(\mathbf{w}) = \Sigma\, \ell_i$.

3. Nếu $m_i \le 0$ thì $\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot y_i \cdot \mathbf{x}_i^T$: cộng với $y = +1$, trừ với $y = -1$. Margin tăng $\eta \|\mathbf{x}_i\|^2$.

4. Duyệt từng mẫu, cập nhật tức thời, lặp qua các epoch; dừng khi $c = 0$ hoặc đạt T_max.

5. Với $\mathbf{w} = \mathbf{0}$, η chỉ làm w tỉ lệ theo, không đổi bộ phân loại.

KIỂM TRA NHANH

Năm câu hỏi

1. Với $\mathbf{w} = [-2{,}5;\ 1;\ -0{,}5]^T$, tính s, m và $J(\mathbf{w})$ trên 6 sinh viên.

2. $\mathbf{w} = [0;\ 1;\ -1]^T$, $\eta = 1$, $\mathbf{x} = [1,\ 2,\ 3]$, $y = +1$. Có cần cập nhật không? Nếu có, tính $\mathbf{w}$ mới và kiểm tra margin tăng đúng $\eta\|\mathbf{x}\|^2$.

3. Vì sao dùng $m \le 0$ thay vì $m < 0$? Điều gì xảy ra nếu $\mathbf{w} = \mathbf{0}$ và dùng $m < 0$?

4. Khởi tạo $\mathbf{w} = \mathbf{0}$ và nhân đôi η. $\mathbf{w}$ cuối và nhãn dự đoán thay đổi thế nào?

5. Với $\mathbf{w} = [-1;\ 4;\ -8]^T$, dự đoán nhãn sinh viên mới học 5 giờ, vắng 1,5 buổi.

GỢI Ý ĐÁP ÁN

Đáp án năm câu

1 $s = [-2{,}5; -1; -1; 3; 4{,}5; 6{,}5]$, $m = [2{,}5; 1; 1; 3; 4{,}5; 6{,}5]$; mọi $m > 0$ nên $J = 0$.

2 $s = -1$, $m = -1 \le 0$ nên cập nhật: $w = [1; 3; 2]^T$, $s' = 13$. Margin tăng $14 = \|x\|^2 = 1 + 4 + 9$.

3 Khi $w = 0$ mọi $m = 0$. Với $m < 0$ không mẫu nào cập nhật, $c = 0$ ở epoch đầu và dừng ngay tại nghiệm vô nghĩa $w = 0$.

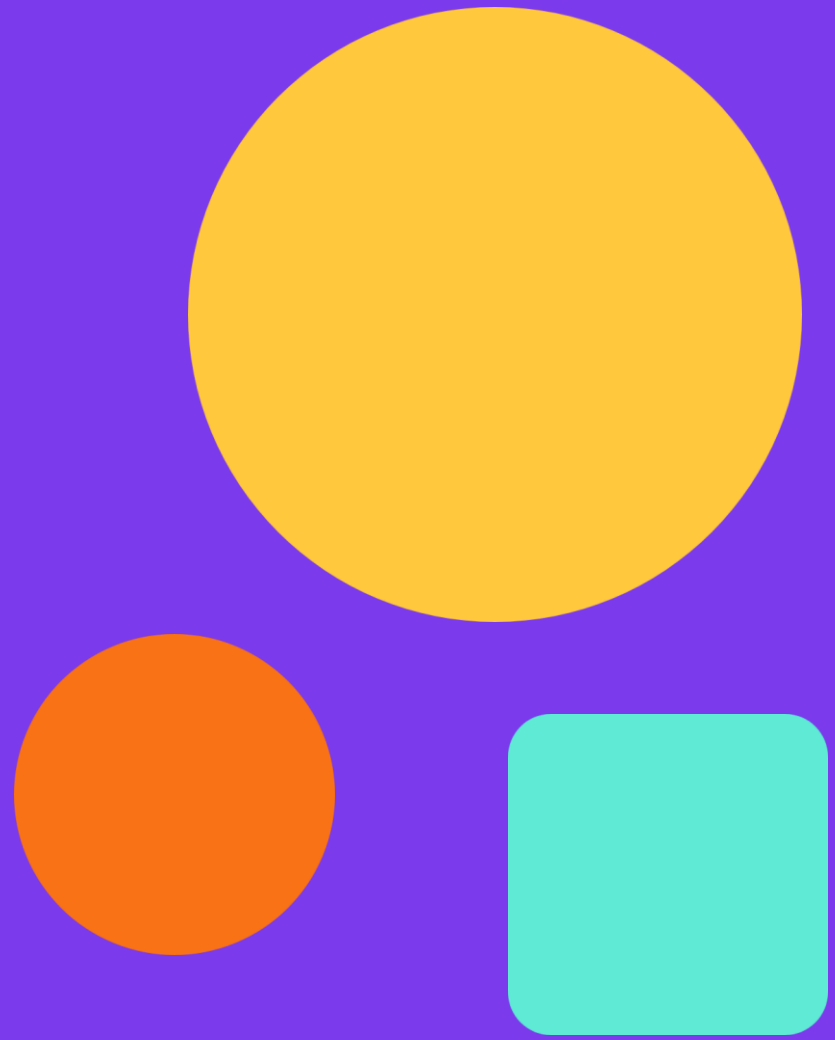
4 w cuối nhân đôi (w^(2η) = 2w^(η)); đường biên và nhãn dự đoán không đổi.

5 $x = [1, 5, 1{,}5]$: $s = -1 + 20 - 12 = 7 \ge 0$, nên $\hat{y} = +1$ (đậu).

HỌC MÁY · BÀI 2 · PERCEPTRON

Phân tích cơ chế học của Perceptron

Mục 4 · Học như thế nào, và khi nào học được?



MỤC TIÊU CỦA MỤC 4

Ba câu hỏi cần trả lời

1

Kết quả huấn luyện có duy nhất không?

Thứ tự mẫu và khởi tạo: mục 4.1.

2

Khi nào chắc chắn thuật toán dừng? Ước lượng được số lần cập nhật không?

Định lý hội tụ: mục 4.2.

3

Điều gì xảy ra khi một đường thẳng không tách được hai lớp?

Bài toán XOR: mục 4.3.

4.1

Thứ tự mẫu và khởi tạo tham số

Cùng dữ liệu, vì sao có nhiều nghiệm khác nhau?

4.1 · THỨ TỰ DUYỆT MẪU

Cập nhật tức thời nên thứ tự có ý nghĩa

Vì sao?

Perceptron sửa w **ngay sau mỗi mẫu**. Thứ tự quyết định mẫu nào được xét khi w đang có giá trị nào.

Thí nghiệm

Dữ liệu 6 sinh viên, **w = 0**, η = 1, chạy **cả 6! = 720 thứ tự** duyệt mẫu.

Câu hỏi: 720 thứ tự có cho cùng một w cuối cùng không?

Hãy đoán trước khi xem kết quả.

4.1 · KẾT QUẢ 720 THỨ TỰ

Luôn đúng, nhưng không duy nhất

100%

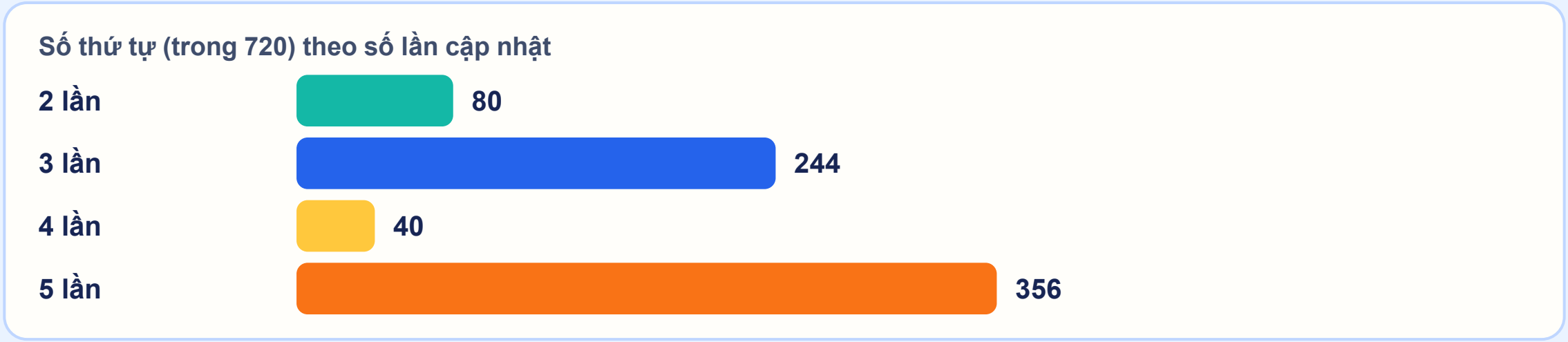
thứ tự dừng và **phân loại đúng cả 6 mẫu**.

2 → 5

số lần cập nhật thay đổi tùy thứ tự.

20

bộ tham số cuối cùng khác nhau.



4.1 · BỐN THỨ TỰ CỤ THỂ

Bốn bộ w khác nhau, đều đúng trên dữ liệu huấn luyện

Thứ tự duyệt	Số lần cập nhật	w cuối cùng
1, 2, 3, 4, 5, 6	5	$[-1; 4; -8]^T$
6, 5, 4, 3, 2, 1	5	$[-1; 6; -11]^T$
1, 4, 2, 5, 3, 6	5	$[-1; 5; -9]^T$
3, 1, 2, 4, 5, 6	2	$[0; 2; -4]^T$

Bốn đường biên khác nhau, cùng phân loại đúng 6 sinh viên huấn luyện.

Sự khác biệt lộ ra khi gặp sinh viên chưa từng thấy.

4.1 · SINH VIÊN MỚI

Học 7 giờ, vắng 3,6 buổi: x = [1, 7, 3,6]

w	Điểm số s = xw	Dự đoán
$[-1; 4; -8]^T$	−1 + 28 − 28,8 = −1,8	−1 (rớt)
$[-1; 6; -11]^T$	−1 + 42 − 39,6 = 1,4	+1 (đậu)
$[-1; 5; -9]^T$	−1 + 35 − 32,4 = 1,6	+1 (đậu)
$[0; 2; -4]^T$	14 − 14,4 = −0,4	−1 (rớt)

Cùng dữ liệu huấn luyện nhưng hai dự đoán “rớt”, hai dự đoán “đậu”.

Perceptron không có cách nào để chọn đường nào tốt hơn.

4.1 · GIÁ TRỊ KHỞI TẠO

Khởi tạo thay đổi quá trình học

Khởi tạo $w^{(0)}$	Số lần cập nhật	w cuối cùng
$[0; 0; 0]^T$	5	$[-1; 4; -8]^T$
$[-2; 1; 0]^T$	3	$[-3; 2; -6]^T$
$[1; 1; 1]^T$	3	$[0; 3; -6]^T$
$[0; -1; 0]^T$	5	$[-1; 3; -8]^T$
$[-2{,}5; 1; -1]^T$	0	$[-2{,}5; 1; -1]^T$

Khởi tạo càng gần một nghiệm đúng thì càng cần ít lần cập nhật.

Dòng cuối: khởi tạo đã đúng hết nên thuật toán dừng ngay, 0 lần cập nhật.

4.1 · KẾT LUẬN

Ba điều cần nhớ

Luôn đúng

Mọi thứ tự và mọi khởi tạo đều cho bộ phân loại đúng toàn bộ dữ liệu huấn luyện (khi dữ liệu tách được).

Không duy nhất

Nghiệm cụ thể phụ thuộc thứ tự và khởi tạo; các nghiệm có thể dự đoán khác nhau.

Động lực mới

Muốn đường biên “tốt nhất” theo tiêu chí rõ ràng, ví dụ cách đều hai lớp, dẫn đến SVM.

Perceptron chỉ đảm bảo “đúng hết”, không đảm bảo “tốt nhất”.

4.2

Điều kiện hội tụ

Dữ liệu phân tách tuyến tính và định lý Novikoff.

4.2 · CÂU HỎI

Thuật toán có chắc chắn dừng không?

Nỗi lo

Mỗi lần cập nhật cải thiện mẫu đang xét nhưng có thể làm xấu mẫu khác. Liệu thuật toán có “giằng co” mãi?

Tin tốt

Có một kết quả toán học đẹp và cụ thể: **định lý hội tụ Perceptron** (Novikoff, 1962).

Ký hiệu: $R = \max_i \|x_i\|$, độ dài lớn nhất của vector đặc trưng (đã gồm số 1 ở đầu).

4.2 · ĐỊNH LÝ HỘI TỤ

Định lý Novikoff

Giả sử tồn tại $\mathbf{w}^*$ với $\|\mathbf{w}^*\| = 1$ và $\gamma > 0$ sao cho

$$y_i \cdot \mathbf{x}_i \mathbf{w}^* \geq \gamma \text{ với mọi } i = 1, \ldots, n$$

Khi đó Perceptron khởi tạo $\mathbf{w} = \mathbf{0}$, với mọi $\eta > 0$, thực hiện **không quá**

$$k \leq R^2 / \gamma^2$$

lần cập nhật.

Điều kiện của định lý chính là dữ liệu phân tách tuyến tính, phát biểu mạnh hơn một chút:
mọi mẫu nằm đúng phía và cách đường biên ít nhất γ.

4.2 · Ý NGHĨA CÁC ĐẠI LƯỢNG

R, γ và w* nói lên điều gì?

R

Độ dài lớn nhất của vector đặc trưng. Dữ liệu “to” hơn thì R lớn hơn.

γ (margin)

Khoảng đệm nhỏ nhất giữa dữ liệu và đường biên w*. Hai lớp càng xa nhau, γ càng lớn.

w*

Một bộ tham số “lý tưởng” tách đúng mọi mẫu; chỉ cần **tồn tại**, thuật toán không biết nó.

Lưu ý: độ dài tính trên vector đầy đủ, cả hệ số chệch w_0 và phần tử 1 của x_i.

Vì vậy γ gần với khoảng cách hình học nhưng không trùng hoàn toàn.

4.2 · VÍ DỤ VỚI DỮ LIỆU SINH VIÊN

Tính cận trên số lần cập nhật

$$\mathbf{w}^* = (1/\sqrt{21})\cdot[-1;\ 2;\ -4]^T$$

$\|\mathbf{w}^*\| = \sqrt{(1+4+16)}/\sqrt{21} = 1$. Với $\mathbf{w} = [-1;\ 2;\ -4]^T$, margin của 6 sinh viên là:

13, 7, 13, 7, 7, 17

$\gamma = 7/\sqrt{21} \approx$ **1,53**

$\|\mathbf{x}_6\|^2 = 1 + 81 + 0 = 82$, nên $R^2 =$ **82**

$$R^2/\gamma^2 = 82\cdot 21/49 \approx 35{,}1$$

Cận trên: không quá 35 lần cập nhật, với mọi thứ tự duyệt.

Thực tế chỉ cần tối đa 5 lần: cận an toàn nhưng khá rộng rãi.

4.2 · CHỨNG MINH (PHÁC THẢO) · BƯỚC 1

w_k ngày càng “thẳng hàng” với w*

Giả sử lần cập nhật thứ k+1 dùng mẫu i, tức $y_i \mathbf{x}_i \mathbf{w}_k \le 0$ và $\mathbf{w}_{\{k+1\}} = \mathbf{w}_k + \eta \cdot y_i \cdot \mathbf{x}_i^T$.

$$(\mathbf{w}^*)^T \mathbf{w}_{\{k+1\}} = (\mathbf{w}^*)^T \mathbf{w}_k + \eta \cdot y_i \cdot \mathbf{x}_i \mathbf{w}^* \ge (\mathbf{w}^*)^T \mathbf{w}_k + \eta\gamma$$

Sau k lần ($\mathbf{w}_o = \mathbf{o}$): $(\mathbf{w}^*)^T \mathbf{w}_k \ge k \cdot \eta \cdot \gamma$

Hình chiếu của w_k lên hướng w* tăng ít nhất tuyến tính theo k.

4.2 · CHỨNG MINH (PHÁC THẢO) · BƯỚC 2

Độ dài của w_k tăng chậm

$$\|w_{\{k+1\}}\|^2 = \|w_k\|^2 + 2\eta \cdot y_i x_i w_k + \eta^2 \|x_i\|^2 \le \|w_k\|^2 + \eta^2 R^2$$

Sau k lần: $\|w_k\|^2 \le k \cdot \eta^2 \cdot R^2$

Dùng $y_i x_i w_k \le 0$ (đúng vì đây là mẫu được cập nhật) và $y_i^2 = 1$.

Độ dài của w_k chỉ tăng cỡ $\sqrt{k}$.

4.2 · CHỨNG MINH (PHÁC THẢO) · BƯỚC 3

Hai đại lượng “va nhau”

$$k \cdot \eta \cdot \gamma \le (\mathbf{w}^*)^T \mathbf{w}_k \le \|\mathbf{w}^*\| \cdot \|\mathbf{w}_k\| \le \eta \cdot R \cdot \sqrt{k}$$

$$\Rightarrow \sqrt{k} \le R/\gamma \Rightarrow k \le R^2/\gamma^2 \ \blacksquare$$

Bất đẳng thức Cauchy–Schwarz và $\|\mathbf{w}^*\| = 1$.

Vế trái

Tăng **tuyến tính** theo k.

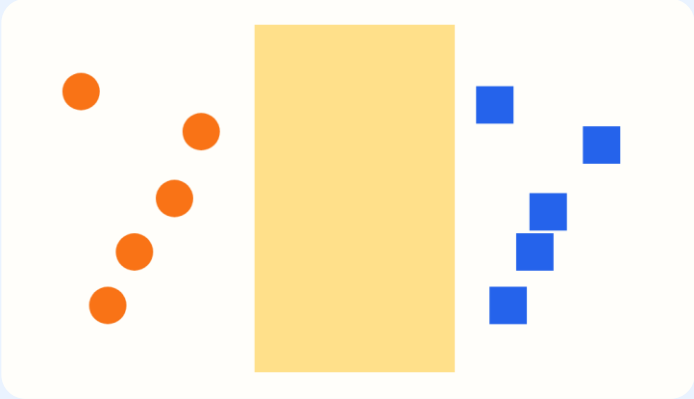
Vế phải

Chỉ tăng cỡ $\sqrt{\mathbf{k}}$.

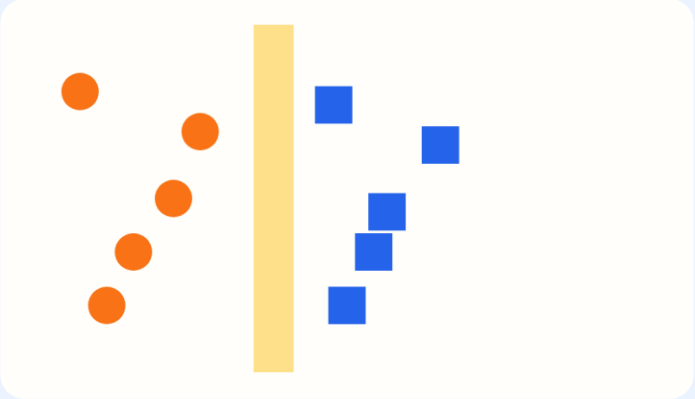
Hình chiếu không thể lớn hơn độ dài của chính nó, nên k không thể tăng mãi.

4.2 · Ý NGHĨA CỦA ĐỊNH LÝ

Cận trên phụ thuộc R/γ, không phụ thuộc số mẫu n



γ lớn: ít lần cập nhật



γ nhỏ: nhiều lần cập nhật

Điều quan trọng không phải có bao nhiêu mẫu, mà là hai lớp cách xa nhau đến mức nào.

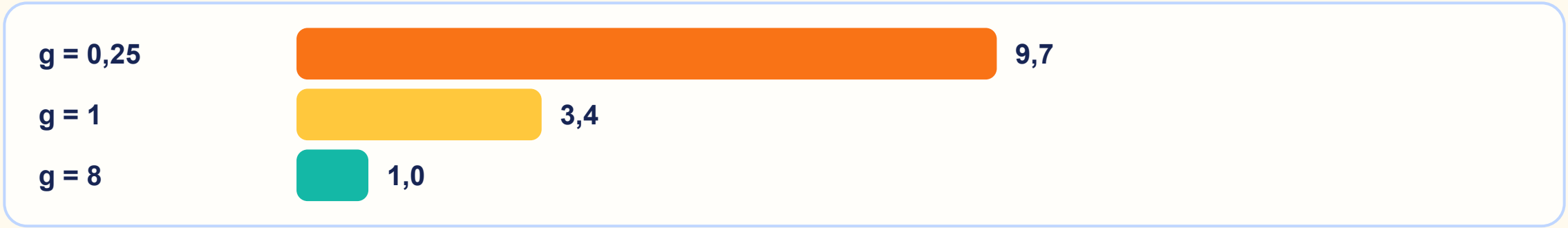
Vùng vàng là “khoảng trống” giữa hai lớp.

4.2 · KIỂM CHỨNG BẰNG MÔ PHỎNG

Khoảng trống càng hẹp, càng nhiều lần cập nhật

Mỗi lớp 50 điểm trong hình vuông 4×4, hai hình vuông cách nhau g theo trục x_1; trung bình 30 bộ dữ liệu ngẫu nhiên ($\mathbf{w}^{(\omega)} = \mathbf{0}$, $\eta = 1$).

g	0,25	0,5	1	2	4	8
Số lần cập nhật TB	9,7	5,6	3,4	2,0	1,5	1,0



4.2 · ĐIỀU KIỆN CẦN VÀ ĐỦ

Tách được ⇔ dừng sau hữu hạn bước

Dữ liệu phân tách tuyến tính được ⇔ Perceptron dừng sau hữu hạn lần cập nhật

Chiều thuận

Định lý Novikoff: tách được thì dừng, không quá R^2/γ^2 lần.

Chiều ngược

Nếu dừng thì epoch cuối không còn mẫu $m \le 0$; chính **w** đó chứng tỏ dữ liệu tách được.

Hội tụ không có nghĩa là tốt nhất.

Thuật toán dừng ngay khi hết mẫu sai, nên đường biên có thể rất sát một số mẫu. Margin lớn nhất là ý tưởng của SVM.

4.3

Trường hợp không hội tụ: XOR

Khi một đường thẳng là không đủ.

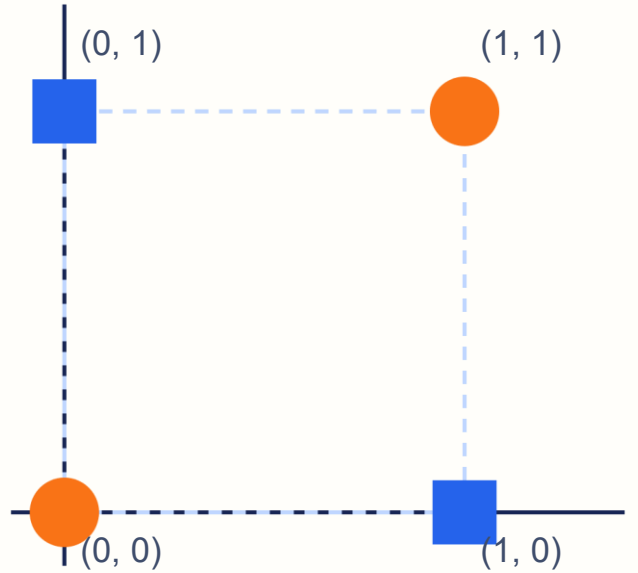
4.3 · BÀI TOÁN XOR

Hai điểm cùng lớp nằm ở hai đỉnh đối diện

x_1	x_2	y
0	0	−1
0	1	+1
1	0	+1
1	1	−1

XOR là gì?

Cho kết quả đúng (+1) khi hai đầu vào **khác nhau**, sai (−1) khi giống nhau.



4.3 · CHỨNG MINH

XOR không phân tách tuyến tính được

Cần w sao cho $y_i \mathbf{x}_i \mathbf{w} > 0$ với cả 4 mẫu:

mẫu 1: $w_0 < 0$

mẫu 2: $w_0 + w_2 > 0$

mẫu 3: $w_0 + w_1 > 0$

mẫu 4: $w_0 + w_1 + w_2 < 0$

Mẫu 2 + mẫu 3:

$$2w_0 + w_1 + w_2 > 0$$

Mẫu 1 + mẫu 4:

$$2w_0 + w_1 + w_2 < 0$$

Hai kết quả mâu thuẫn: không tồn tại w nào.

Kết luận đúng với mọi mô hình có đường biên là siêu phẳng, không riêng Perceptron.

4.3 · PERCEPTRON CHẠY TRÊN XOR

w = 0, η = 1, thứ tự 1 → 4: cả 4 mẫu đều cập nhật

Mẫu	x_i	y_i	s	m	w sau
1	[1, 0, 0]	−1	0	0	$[-1; 0; 0]^T$
2	[1, 0, 1]	+1	−1	−1	$[0; 0; 1]^T$
3	[1, 1, 0]	+1	0	0	$[1; 1; 1]^T$
4	[1, 1, 1]	−1	3	−3	$[0; 0; 0]^T$

Cuối epoch 1, w quay về đúng 0: trạng thái ban đầu.

Epoch 2 lặp lại y hệt epoch 1, và cứ thế mãi: w quay vòng, không bao giờ dừng.

4.3 · SỐ LẦN CẬP NHẬT THEO EPOCH

Dữ liệu tách được hạ về 0, XOR luôn là 4

Dữ liệu 6 sinh viên



XOR



Dữ liệu không tách được: luôn còn ít nhất một mẫu m ≤ 0 ở mọi epoch, nên c = 0 không bao giờ xảy ra.

Vì vậy cần điều kiện dừng thứ hai: số epoch tối đa T_max.

4.3 · TỐT NHẤT CÓ THỂ LÀ BAO NHIÊU?

Độ chính xác tối đa trên XOR là 3/4 = 75%

Ví dụ đúng 3 mẫu

$\mathbf{w} = [-0{,}5;\ 1;\ 1]^T$: $s = -0{,}5 + x_1 + x_2$. Mẫu 1, 2, 3 đúng, mẫu 4 sai.

Vấn đề

Perceptron kinh điển dao động giữa nhiều nghiệm; điểm dừng tại T_max có thể là nghiệm kém.

Thuật toán túi (pocket)

1. Chạy Perceptron như thường, nhưng giữ trong "túi" **w** có **ít mẫu sai nhất** từ trước đến giờ.
2. w hiện tại tốt hơn $\mathbf{w}_{túi}$ thì thay $\mathbf{w}_{túi}$.
3. Khi đạt T_max, trả về $\mathbf{w}_{túi}$ chứ không phải w cuối.

4.3 · HƯỚNG KHẮC PHỤC

Đổi cách biểu diễn: thêm đặc trưng x_1x_2

$$\mathbf{x}' = [1, x1, x2, x1x2]$$

$$\mathbf{w}' = [-0{,}5;\ 1;\ 1;\ -2]^T$$

Điểm số $\mathbf{s} = \mathbf{X}'\mathbf{w}' = [-0{,}5;\ 0{,}5;\ 0{,}5;\ -0{,}5]^T$

$\hat{\mathbf{y}} = [-1, +1, +1, -1] = \mathbf{y}$

$\mathbf{m} = [0{,}5;\ 0{,}5;\ 0{,}5;\ 0{,}5]$, mọi margin đều dương.

Mô hình vẫn tuyến tính theo x′, nhưng trong không gian (x_1, x_2) đường biên là đường cong.

Hạn chế: phải tự đoán ra đặc trưng phù hợp.

4.3 · HƯỚNG TỔNG QUÁT

Ghép nhiều Perceptron thành nhiều lớp

Ý tưởng

Các lớp đầu **tự học** đặc trưng hữu ích cho lớp sau.

XOR bằng logic

Kết hợp các phép Perceptron đơn làm được: "x_1 HOẶC x_2" và "KHÔNG PHẢI (x_1 VÀ x_2)".

Lịch sử

Năm 1969, Minsky và Papert chỉ ra giới hạn của Perceptron đơn, làm giảm quan tâm đến mạng nơ-ron một thời gian.

Trong thực tế dữ liệu có nhiễu nên hiếm khi tách tuyến tính hoàn hảo.

Perceptron thuần thường không dừng: cần T_max, thuật toán túi, hoặc mô hình có mất mát trơn như Logistic Regression.

TÓM TẮT MỤC 4

Sáu ý cần nhớ

1. Kết quả không duy nhất: thứ tự và khởi tạo cho w khác nhau, đều đúng trên dữ liệu huấn luyện nhưng dự đoán mới có thể khác.

2. Định lý hội tụ: $k \le R^2/\gamma^2$, không phụ thuộc số mẫu n.

3. Chứng minh: $(\mathbf{w}^*)^T\mathbf{w}_k \ge k\eta\gamma$ (tăng tuyến tính) và $\|\mathbf{w}_k\|^2 \le k\eta^2R^2$ (tăng chậm).

4. Tách được ⇔ Perceptron dừng sau hữu hạn bước; hội tụ không có nghĩa là tốt nhất.

5. XOR không tách được: Perceptron quay vòng mãi, nên cần T_max, có thể dùng thuật toán túi.

6. Khắc phục giới hạn: thêm đặc trưng phi tuyến hoặc ghép nhiều lớp (mạng nơ-ron).

KIỂM TRA NHANH

Sáu câu hỏi

1. $\mathbf{w} = [0; 1; 0]^T$, $\eta = 1$, $\mathbf{x} = [1, 2, 1]$, $y = -1$. Có cần cập nhật không? Nếu có, tính $\mathbf{w}$ mới và kiểm tra $m' = m + \eta\|\mathbf{x}\|^2$.

2. $R = 10$, $\gamma = 0{,}5$: cận trên số lần cập nhật? Nếu γ tăng gấp đôi, R giữ nguyên thì sao?

3. Kiểm tra $\mathbf{w} = [-1{,}5; 1; 1]^T$ phân loại đúng cổng AND và tính vector margin m.

4. Vì sao hai thứ tự duyệt khác nhau cho hai w khác nhau dù cùng đúng 100% dữ liệu huấn luyện? Hệ quả với dữ liệu mới?

5. Vì sao nhãn nhiễu thường làm Perceptron không dừng? Nên làm gì?

6. Với $\mathbf{x}' = [1, x_1, x_2, x_1x_2]$, $\mathbf{w}' = [-0{,}5; 1; 1; -2]^T$: tính điểm số và nhãn dự đoán cho $(x_1, x_2) = (1, 0)$.

GỢI Ý ĐÁP ÁN

Đáp án sáu câu

1 $s = 2$, $m = -2 \le 0$ nên cập nhật: $\mathbf{w} = [-1; -1; -1]^T$, $s' = -4$, $m' = 4 = -2 + (1 + 4 + 1)$.

2 $R^2/\gamma^2 = 100/0{,}25 = 400$. γ gấp đôi thì cận giảm còn 1/4, tức 100.

3 $\mathbf{s} = [-1{,}5; -0{,}5; -0{,}5; 0{,}5]$, $\mathbf{y} = [-1; -1; -1; +1]$, $\mathbf{m} = [1{,}5; 0{,}5; 0{,}5; 0{,}5]$: đều dương nên đúng cả 4 mẫu.

4 Mỗi thứ tự gặp mẫu sai ở thời điểm khác nhau nên cập nhật khác nhau. Có vô số đường biên đúng, thuật toán dừng ở một đường bất kỳ; các đường này có thể dự đoán khác nhau trên dữ liệu mới.

5 Mẫu bị gán sai nhãn làm dữ liệu không còn tách được, nên luôn còn mẫu $m \le 0$ ở mọi epoch. Cần T_max, thuật toán túi, hoặc mô hình chịu nhiễu như Logistic Regression.

6 $\mathbf{x}' = [1, 1, 0, 0]$: $s = -0{,}5 + 1 = 0{,}5 \ge 0$ nên $\hat{y} = +1$, đúng với nhãn XOR của (1, 0).

HỌC MÁY · BÀI 2 · PERCEPTRON

Cài đặt và đánh giá mô hình Perceptron

Mục 5 · Từ công thức đến chương trình chạy được

NỘI DUNG CỦA MỤC 5

Ba việc cần làm

1

Cài đặt Perceptron từ đầu bằng NumPy

Kiểm tra khớp với ví dụ tính tay: mục 5.1.

2

Đánh giá mô hình học được tốt đến đâu

Tập huấn luyện và kiểm tra, accuracy, ma trận nhầm lẫn, precision, recall, F1: mục 5.2.

3

So sánh với scikit-learn

Thư viện làm gì bên trong: mục 5.3.

QUY ƯỚC CHUYỂN TỪ TOÁN SANG MÃ

Mỗi công thức có đúng một dòng mã

Ký hiệu toán	Trong mã Python
Ma trận đặc trưng **X** (n × (d+1), cột đầu bằng 1)	Xb (kết quả của them_bias)
Vector tham số **w**	w
Vector đặc trưng $\mathbf{x}_i$	Xb[i]
Điểm số $\mathbf{s}_i = \mathbf{x}_i\mathbf{w}$	Xb[i] @ w
Vector điểm số $\mathbf{s} = \mathbf{X}\mathbf{w}$	Xb @ w
Nhãn dự đoán ŷ = sign(s)	np.where(s >= 0, 1, -1)
Margin $m_i = y_i \cdot s_i$	y[i] * (Xb[i] @ w)
Cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot y_i \cdot \mathbf{x}_i^T$	w = w + eta * y[i] * Xb[i]

5.1

Cài đặt Perceptron từ đầu

Khoảng 20 dòng NumPy.

5.1 · HAI HÀM NHỎ

Thêm hệ số chệch và dự đoán

PYTHON

```
import numpy as np

def them_bias(X):
    # (n, d) -> (n, d+1)
    return np.hstack([np.ones((X.shape[0], 1)), X])

def du_doan(Xb, w):
    s = Xb @ w
    return np.where(s >= 0, 1, -1)
```

them_bias

Gắn cột toàn số 1 ở đầu để w_0 trở thành một phần tử của w.

du_doan

Một phép nhân ma trận Xb @ w, không cần vòng lặp. Quy ước sign(0) = +1.

5.1 · HÀM HUẤN LUYỆN

huan_luyen: đối chiếu từng dòng với thuật toán mục 3.4

PYTHON

```python
def huan_luyen(Xb, y, eta=1.0, t_max=100):
    n, d1 = Xb.shape
    w = np.zeros(d1)              # khởi tạo w = 0
    lich_su = []                  # số cập nhật mỗi epoch
    quy_dao = []                  # w sau mỗi lần cập nhật
    for epoch in range(t_max):
        so_cap_nhat = 0
        for i in range(n):
            m = y[i] * (Xb[i] @ w)    # margin
            if m <= 0:
                w = w + eta * y[i] * Xb[i]
                quy_dao.append(w.copy())
                so_cap_nhat += 1
        lich_su.append(so_cap_nhat)
        if so_cap_nhat == 0:      # hội tụ
            break
    return w, lich_su, quy_dao
```

Ba kết quả trả về

w cuối cùng, lich_su (số cập nhật từng epoch) và quy_dao (w sau mỗi lần cập nhật, để vẽ hình).

Dừng khi nào

so_cap_nhat == 0 hoặc đủ t_max epoch.

5.1 · KIỂM TRA TRÊN 6 SINH VIÊN

Chạy đúng dữ liệu của mục 3.5

PYTHON

```
X_sv = np.array([[2, 4], [3, 3], [4, 5],
                 [6, 1], [8, 2], [9, 0]], dtype=float)
y_sv = np.array([-1, -1, -1, 1, 1, 1])
Xb_sv = them_bias(X_sv)
w, lich_su, quy_dao = huan_luyen(Xb_sv, y_sv)
print("w =", w)
print("So cap nhat moi epoch:", lich_su)
print("y_hat =", du_doan(Xb_sv, w))
print("margin m =", y_sv * (Xb_sv @ w))
```

KẾT QUẢ IN RA

```
w = [-1.  4. -8.]
So cap nhat moi epoch: [2, 2, 1, 0]
y_hat = [-1 -1 -1  1  1  1]
margin m = [25. 13. 25. 15. 15. 35.]
```

Khớp hoàn toàn với tay: w = [−1; 4; −8]^T, số cập nhật 2, 2, 1, 0, m = [25, 13, 25, 15, 15, 35].

5 dòng của quy_dao chính là 5 lần cập nhật ở bảng mục 3.5.

5.1 · TRỰC QUAN HÓA QUÁ TRÌNH HỌC

Vẽ đường biên sau mỗi lần cập nhật

PYTHON (RÚT GỌN)

```
def ve_duong_bien(ax, X, y, w, x1_max=10, x2_max=6):
    u, v = np.meshgrid(np.linspace(0, x1_max, 200),
                       np.linspace(-1, x2_max, 200))
    s = w[0] + w[1] * u + w[2] * v
    ax.contourf(u, v, np.where(s >= 0, 1, -1), ...)
    ax.contour(u, v, s, levels=[0], colors="k")
    ax.scatter(*X[y == -1].T, c="tab:red")
    ax.scatter(*X[y == 1].T, c="tab:blue", marker="s")
```

Ý tưởng

Đường biên là tập điểm có $s = w_0 + w_1x_1 + w_2x_2 = 0$; hai vùng tô màu là hai nhãn dự đoán.

Năm biểu đồ

Gọi hàm này cho từng w trong quy_dao để thấy đường biên xoay dần như ở mục 3.5.

5.2

Huấn luyện và đánh giá trên dữ liệu lớn hơn

Mô hình học được tốt đến đâu?

5.2 · DỮ LIỆU

400 sinh viên mô phỏng, hai lớp chồng lấn

PYTHON

```
rng = np.random.default_rng(42)
n_lop = 200
dau = np.c_[rng.normal(8, 2.2, n_lop),
            rng.normal(1.5, 1.2, n_lop)]   # +1
rot = np.c_[rng.normal(4, 2.2, n_lop),
            rng.normal(3.5, 1.5, n_lop)]   # -1
X = np.clip(np.vstack([dau, rot]), 0, None)
y = np.r_[np.ones(n_lop),
          -np.ones(n_lop)].astype(int)
```

Hai đặc trưng

x_1 = số giờ tự học, x_2 = số buổi vắng.

Chồng lấn

Đậu: học nhiều, vắng ít; rớt thì ngược lại, nhưng hai nhóm lấn vào nhau như dữ liệu thật.

5.2 · TẬP HUẤN LUYỆN VÀ TẬP KIỂM TRA

Vì sao phải chia dữ liệu?

Nếu đánh giá trên chính dữ liệu đã học

Ta chỉ đo mô hình **nhớ** dữ liệu cũ tốt đến đâu. Giống học sinh ôn đúng đề rồi thi lại đúng đề đó.

Chia ngay từ đầu

Huấn luyện 70%: để học w.
Kiểm tra 30%: giấu đi, chỉ dùng một lần ở cuối.

Quy tắc vàng: không dùng tập kiểm tra để chọn tham số, chọn T_max hay bất kỳ quyết định nào khi huấn luyện.

Vi phạm thì ước lượng trên tập kiểm tra sẽ quá lạc quan.

5.2 · MÃ CHIA DỮ LIỆU

Xáo trộn rồi chia 70/30

PYTHON

```
def chia_train_test(X, y, ti_le_test=0.3, seed=0):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y))
    n_test = int(len(y) * ti_le_test)
    return (X[idx[n_test:]], X[idx[:n_test]],
            y[idx[n_test:]], y[idx[:n_test]])

X_tr, X_te, y_tr, y_te = chia_train_test(X, y)
Xb_tr, Xb_te = them_bias(X_tr), them_bias(X_te)
```

280

mẫu huấn luyện (149 sinh viên đậu).

120

mẫu kiểm tra (51 sinh viên đậu).

5.2 · HUẤN LUYỆN THUẦN TÚY

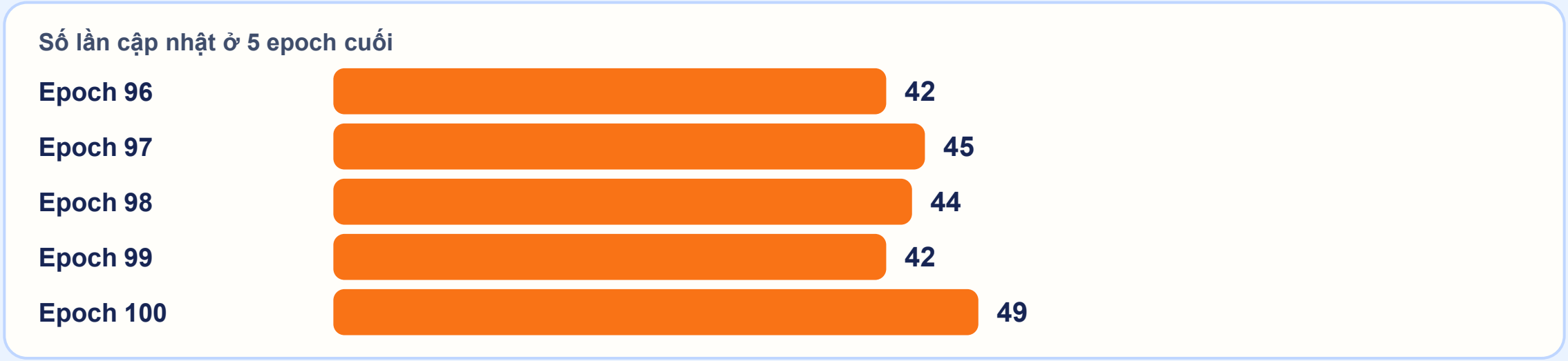
Dữ liệu chồng lấn nên không hội tụ

PYTHON

```
w_1, lich_su_1, _ = huan_luyen(Xb_tr, y_tr, t_max=100)
print("so epoch da chay:", len(lich_su_1),
      "| so cap nhat 5 epoch cuoi:", lich_su_1[-5:])
```

KẾT QUẢ IN RA

```
so epoch da chay: 100 |
so cap nhat 5 epoch cuoi:
[42, 45, 44, 42, 49]
```



5.2 · THUẬT TOÁN TÚI

Giữ lại w có ít mẫu sai nhất

PYTHON (RÚT GỌN)

```
def huan_luyen_tui(Xb, y, eta=1.0, t_max=100, seed=0):
    n = Xb.shape[0]
    rng = np.random.default_rng(seed)
    w = np.zeros(Xb.shape[1]); w_tui = w.copy()
    loi_tui = np.sum(du_doan(Xb, w) != y)
    for epoch in range(t_max):
        for i in rng.permutation(n):    # xáo trộn
            if y[i] * (Xb[i] @ w) <= 0:
                w = w + eta * y[i] * Xb[i]
                loi = np.sum(du_doan(Xb, w) != y)
                if loi < loi_tui:       # tốt hơn túi
                    w_tui, loi_tui = w.copy(), loi
    return w, w_tui
```

Ba bước

Cập nhật như thường; sau mỗi lần cập nhật đếm số mẫu sai; nếu ít hơn mọi lần trước thì cất w vào túi.

Xáo trộn

Đổi thứ tự mẫu ở mỗi epoch để kết quả không phụ thuộc cách sắp xếp dữ liệu.

5.2 · KẾT QUẢ THUẬT TOÁN TÚI

w hiện tại dao động mạnh, w túi chỉ giảm

69 / 280

mẫu sai của w ở epoch cuối (dao động từ khoảng 24 đến hơn 100 mẫu sai giữa các epoch).

23 / 280

mẫu sai của w_túi: chỉ giảm, không bao giờ tăng.

KẾT QUẢ IN RA

```
w_cuoi = [-56.      12.73  -21.572] | w_tui = [-52.      18.135 -19.169]
loi tren train: w_cuoi = 69 | w_tui = 23 / 280
```

Nếu dừng ở một epoch bất kỳ, ta có thể nhận về một w rất tệ; w_túi thì luôn là tốt nhất từ trước đến giờ.

5.2 · ACCURACY

Tỉ lệ mẫu được dự đoán đúng

$$\mathrm{Accuracy} = (1/n) \cdot \Sigma\, \mathbb{1}[\hat{y}_i = y_i] = (1/n) \cdot \Sigma\, (1 + y_i \hat{y}_i)/2$$

Nhãn ±1: tích $y_i \hat{y}_i$ = +1 khi đúng, −1 khi sai. Trong mã: np.mean(y == y_hat).

Bộ tham số	Huấn luyện	Kiểm tra
w thuần túy (không xáo trộn, epoch 100)	88,2%	85,8%
w epoch cuối (có xáo trộn)	75,4%	82,5%
w_túi (thuật toán túi)	91,8%	89,2%

5.2 · BA NHẬN XÉT

Đọc bảng accuracy

Túi tốt và ổn định nhất

w epoch cuối dao động nên chỉ đạt 75,4% trên tập huấn luyện.

Chênh lệch nhỏ là bình thường

Kiểm tra 89,2% so với huấn luyện 91,8%. Nếu chênh rất lớn thì mô hình đang **học thuộc** (overfit).

Sai số ngẫu nhiên

Tập kiểm tra chỉ 120 mẫu nên 89,2% có thể đổi vài điểm phần trăm nếu chia khác.

Luôn báo cáo kết quả trên tập kiểm tra, không phải tập huấn luyện.

5.2 · MA TRẬN NHẦM LẪN

Sai kiểu nào? Hai loại sai khác nhau

Lớp dương là “đậu” (y = +1). Hàng là nhãn thực, cột là nhãn dự đoán.

TN

đoán rớt, đúng

FP

báo nhầm: rớt mà đoán đậu

FN

bỏ sót: đậu mà đoán rớt

TP

đoán đậu, đúng

C = [[TN, FP], [FN, TP]] · Accuracy = (TP + TN) / (TP + TN + FP + FN)

5.2 · MA TRẬN NHẦM LẪN TRÊN TẬP KIỂM TRA

C = [[59, 10], [3, 48]]

PYTHON

```
def ma_tran_nham_lan(y, y_hat):
    tp = np.sum((y == 1) & (y_hat == 1))
    tn = np.sum((y == -1) & (y_hat == -1))
    fp = np.sum((y == -1) & (y_hat == 1))
    fn = np.sum((y == 1) & (y_hat == -1))
    return np.array([[tn, fp], [fn, tp]])
```

59 TN	**10** FP
3 FN	**48** TP

Đoán đúng 59 sinh viên rớt và 48 sinh viên đậu; báo nhầm 10, bỏ sót 3.

Accuracy = (48 + 59)/120 = 107/120 ≈ 89,2%. Mô hình mắc lỗi báo nhầm nhiều hơn bỏ sót.

5.2 · PRECISION, RECALL VÀ F1

Ba chỉ số cho lớp dương

Precision

TP/(TP+FP). Trong số sinh viên được dự đoán đậu, bao nhiêu thật sự đậu? Thấp nghĩa là hay **báo nhầm**.

Recall

TP/(TP+FN). Trong số sinh viên thật sự đậu, phát hiện được bao nhiêu? Thấp nghĩa là hay **bỏ sót**.

F1

2TP/(2TP+FP+FN). Trung bình điều hòa, chỉ cao khi cả hai cùng cao.

$$Precision = 48/58 \approx 0{,}828 \cdot Recall = 48/51 \approx 0{,}941 \cdot F1 = 96/109 \approx 0{,}881$$

Phát hiện 94,1% sinh viên đậu, nhưng chỉ 82,8% dự đoán “đậu” là đúng.

5.2 · KHI NÀO CẦN CHỈ SỐ NÀO?

Tầm quan trọng của hai loại sai phụ thuộc bài toán

Tầm soát bệnh

Bỏ sót bệnh nhân (FN) nguy hiểm hơn báo nhầm, nên ưu tiên **recall** cao.

Lọc thư rác

Báo nhầm thư quan trọng là thư rác (FP) gây thiệt hại hơn, nên ưu tiên **precision** cao.

Không có chỉ số “tốt nhất” cho mọi bài toán: hãy hỏi lỗi nào đắt hơn.

5.2 · CẠM BẪY CỦA ACCURACY

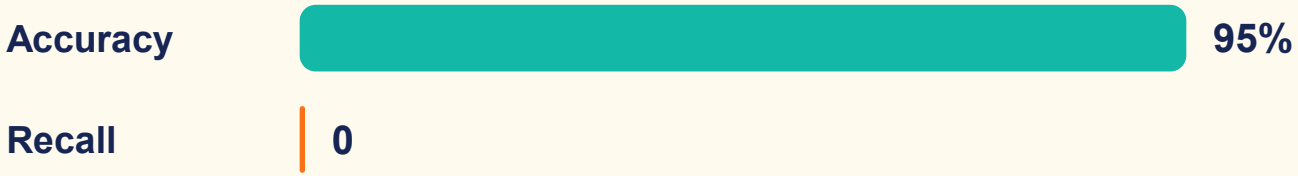
Dữ liệu lệch lớp: mô hình “lười” vẫn đạt 95%

Tình huống

100 mẫu: 95 âm, 5 dương. Mô hình luôn dự đoán −1: TN = 95, FN = 5, TP = FP = 0.

Kết quả

Accuracy = 95/100 = **95%** nhưng Recall = 0/(0+5) = **0**.



Với dữ liệu lệch lớp, luôn xem ma trận nhầm lẫn, precision và recall, đừng chỉ nhìn accuracy.

5.3

So sánh với scikit-learn

Thư viện làm gì bên trong?

5.3 · LỚP PERCEPTRON CỦA SCIKIT-LEARN

Cùng thuật toán, kết quả trùng khớp

PYTHON

```
from sklearn.linear_model import Perceptron
clf = Perceptron(eta0=1.0, max_iter=20,
                 tol=None, shuffle=False)
clf.fit(X_sv, y_sv)
print("intercept_ =", clf.intercept_,
      "| coef_ =", clf.coef_)
```

KẾT QUẢ IN RA

```
intercept_ = [-1.]
coef_ = [[ 4. -8.]]
```

eta0, max_iter

η và T_max.

tol=None

Tắt dừng sớm của thư viện.

shuffle=False

Không xáo trộn, để so với mã của ta.

5.3 · KIỂM CHỨNG TRÊN DỮ LIỆU LỚN

Hai kết quả trùng nhau

PYTHON

```
clf = Perceptron(eta0=1.0, max_iter=100,
                 tol=None, shuffle=False)
clf.fit(X_tr, y_tr)
print("sklearn: intercept_ =", clf.intercept_,
      "coef_ =", clf.coef_)
print("tu cai dat: w =", w_1)
```

KẾT QUẢ IN RA

```
sklearn: intercept_ = [-65.]
  coef_ = [[ 27.118 -27.472]]
tu cai dat: w = [-65.  27.118 -27.472]
```

Perceptron của scikit-learn chính là thuật toán ta đã viết.

Cập nhật khi $m \le 0$, bước $\eta \cdot y_i \cdot x_i^T$, khởi tạo $w = 0$, duyệt theo epoch.

5.3 · NHỮNG KHÁC BIỆT CẦN BIẾT

Mã tự cài đặt và scikit-learn

Điểm	Mã tự cài đặt	scikit-learn
Hệ số chệch	Phần tử đầu w_0 của w (cần them_bias)	Lưu riêng trong intercept_; coef_ chứa w_1…w_d
Mẫu có s = 0	Dự đoán +1 (sign(0) = +1)	Dự đoán lớp thứ nhất (−1), vì chỉ gán lớp dương khi s > 0
Thứ tự duyệt	Cố định (hoặc xáo trộn nếu cài đặt)	Mặc định xáo trộn (shuffle=True)
Điều kiện dừng	c = 0 hoặc T_max	Mặc định dừng sớm khi cải thiện nhỏ hơn tol (1e-3)
Nhãn	{−1, +1}	Hai nhãn bất kỳ, lưu trong classes_

5.3 · SO SÁNH ĐỘ CHÍNH XÁC

Ba mô hình, kết quả gần như bằng nhau

Mô hình	Huấn luyện	Kiểm tra
Perceptron tự cài đặt (thuật toán túi)	91,8%	89,2%
Perceptron của scikit-learn (mặc định)	90,7%	89,2%
Hồi quy tuyến tính + ngưỡng 0	90,7%	89,2%

Không mâu thuẫn với mục 1.2: dữ liệu khá cân bằng, không có ngoại lai cực đoan nên hồi quy rồi đặt ngưỡng vẫn tạm ổn.

Điểm yếu của hồi quy xuất hiện khi có mẫu “rất đúng” hoặc ngoại lai.

5.3 · ĐÁNH GIÁ BẰNG HÀM CÓ SẴN

classification_report khớp với phép tính tay

PYTHON

```
from sklearn.metrics import (confusion_matrix,
                             classification_report)
print(confusion_matrix(y_te, y_hat_te,
      labels=[-1, 1]))
print(classification_report(y_te, y_hat_te,
      labels=[-1, 1], target_names=["rot", "dau"],
      digits=3))
```

KẾT QUẢ IN RA

```
[[59 10]
 [ 3 48]]
              precision recall f1-score support
  rot       0.952  0.855   0.901       69
  dau       0.828  0.941   0.881       51
accuracy                    0.892      120
```

Dòng “dau”: precision 0,828, recall 0,941, F1 0,881, đúng bằng tính tay.

Dòng “rot” tính cho lớp âm: precision 59/62 ≈ 0,952, recall 59/69 ≈ 0,855.

5.3 · LỜI KHUYÊN THỰC HÀNH

Hai việc nên làm với dữ liệu thật

Chuẩn hóa đặc trưng

Ví dụ bằng StandardScaler, để các đặc trưng có thang đo tương đương.

Báo cáo trên tập kiểm tra

Không báo cáo kết quả trên tập huấn luyện.

Cài đặt từ đầu giúp hiểu; dùng thư viện giúp làm việc nhanh và ít lỗi.

TÓM TẮT MỤC 5

Năm ý cần nhớ

1. Perceptron cài đặt bằng khoảng 20 dòng NumPy; mỗi công thức một dòng mã, khớp hoàn toàn với ví dụ tính tay.
2. Chia dữ liệu thành tập huấn luyện (để học) và tập kiểm tra (đánh giá, dùng cuối cùng).
3. Dữ liệu thật chồng lấn nên Perceptron không hội tụ; thuật toán túi giữ w tốt nhất và ổn định hơn.
4. Accuracy có thể gây hiểu lầm; ma trận nhầm lẫn cho biết kiểu sai, từ đó tính precision, recall, F1.
5. Perceptron của scikit-learn cùng thuật toán; khác ở hệ số chệch, thứ tự duyệt mặc định và điều kiện dừng.

KIỂM TRA NHANH

Sáu câu hỏi

1. Vì sao không nên đánh giá mô hình trên chính tập đã huấn luyện?
2. **C** = [[40, 5], [10, 45]] (hàng: thực −1, +1; cột: dự đoán −1, +1). Tính accuracy, precision, recall, F1 cho lớp dương.
3. Mô hình tầm soát bệnh accuracy 98% trên dữ liệu có 2% người bệnh. Vì sao chưa đủ kết luận mô hình tốt? Cần xem chỉ số nào?
4. Đổi if m <= 0 thành if m < 0 với w = np.zeros(d1): điều gì xảy ra?
5. Viết mã dự đoán cho sinh viên mới học 6 giờ, vắng 2 buổi với **w** = $[-1; 4; -8]^T$. Kết quả?
6. Vì sao scikit-learn nhận X không có cột 1 còn mã tự cài đặt phải gọi them_bias?

GỢI Ý ĐÁP ÁN

Đáp án sáu câu

1 Mô hình đã “nhìn thấy” dữ liệu huấn luyện nên accuracy ở đó thường cao hơn và không phản ánh khả năng tổng quát hóa; tập kiểm tra được giấu đi nên cho ước lượng trung thực.

2 TN = 40, FP = 5, FN = 10, TP = 45. Accuracy = 85/100 = 85%; precision = 45/50 = 0,90; recall = 45/55 ≈ 0,818; F1 = 90/105 ≈ 0,857.

3 Mô hình luôn đoán “không bệnh” cũng đạt 98% nhưng bỏ sót mọi bệnh nhân (recall = 0). Cần xem recall (và precision, ma trận nhầm lẫn) của lớp bệnh.

4 Với w = 0 mọi m = 0 nên m < 0 không bao giờ đúng: không cập nhật lần nào, epoch đầu c = 0 và dừng ngay tại w = 0, nghiệm vô nghĩa.

5 x = np.array([1, 6, 2]); w = np.array([-1, 4, -8]); s = x @ w; y_hat = 1 if s >= 0 else -1. s = −1 + 24 − 16 = 7 ≥ 0 nên ŷ = +1 (đậu).

6 scikit-learn tự quản lý hệ số chệch (fit_intercept=True) và lưu w_0 ở intercept_; mã tự cài đặt gộp w_0 vào w nên cần cột 1 để xw tự cộng w_0.